

OWASP Juice Shop VAPT Report

Vulnerability Assessment and Penetration Testing

Security Assessment Report

Prepared By

Mohan Routhu

Target: OWASP Juice Shop

Environment: Localhost

Assessment Type: VAPT

September 2026

Contents

1	Executive Summary	1
2	Introduction	4
3	Assessment Approach	5
3.1	Project Objectives	5
3.2	Scope and Rules of Engagement	6
3.2.1	Assessment Scope	6
3.2.2	In-Scope Components	6
3.2.3	Out-of-Scope Components	6
3.2.4	Rules of Engagement	7
3.2.5	Assessment Authorization	7
3.3	VAPT Methodology	7
3.4	Tools and Technologies	7
4	Phase 1 – Planning and Scoping	9
4.1	Phase Objectives	9
4.2	Assessment Methodology	9
4.3	Expected Deliverables	10
4.4	Phase 1 Summary	10
4.5	Phase Completion	10
5	Phase 2 – Reconnaissance	11
5.1	Objective	11
5.2	Target Identification	11
5.3	Port and Service Discovery	12
5.4	Technology Identification	12
5.5	robots.txt Analysis	13
5.6	HTTP Request and Response Analysis	14
5.7	Directory and Endpoint Discovery	14
5.8	OWASP ZAP Spider	15
5.9	OWASP ZAP AJAX Spider	16
5.10	Reconnaissance Summary	16
5.11	Phase 3 Preparation	17

6	Phase 3 – Vulnerability Assessment	18
6.1	Objective	18
6.2	Scanning Methodology	18
6.3	ZAP Scan Evidence	18
6.4	Scan Results	19
6.5	Findings Selected for Manual Validation	20
6.6	Phase 3 to Phase 4	20
6.7	Note	20
7	Phase 4 – Manual Verification and Proof of Concept	21
7.1	Broken Access Control	21
7.2	Insecure Direct Object Reference	24
7.3	Authentication Vulnerabilities	26
7.4	SQL Injection	30
7.5	Cross-Site Scripting	36
7.6	Cross-Site Request Forgery	39
7.7	Open Redirect	43
8	Phase 5 – Post-Exploitation and Impact Analysis	46
9	Phase 6 – Reporting and Risk Assessment	49
9.1	Vulnerability Findings Overview	49
9.2	Risk Summary	50
10	Phase 7 – Remediation and Retesting	53
10.1	Remediation Recommendations	53
10.2	Retesting	56
11	Conclusion	59

1 Executive Summary

Assessment Overview

The assessment began with the definition of the testing scope, objectives, rules of engagement, and target environment. Reconnaissance activities were then performed to identify application functionality, endpoints, parameters, and other accessible components.

Automated security testing was performed using tools such as OWASP ZAP and FFUF, while SQLMap was used where appropriate for SQL Injection assessment. The identified potential vulnerabilities were then manually verified using techniques including crafted HTTP requests and Burp Suite.

Only vulnerabilities that were successfully validated and supported by appropriate evidence were included as confirmed findings in this report.

Confirmed Findings

A total of eight vulnerabilities were confirmed during the assessment.

Finding ID	Vulnerability	Severity
BAC	Broken Access Control – Privilege Escalation via Registration	Critical
IDOR	Insecure Direct Object Reference – Unauthorized Basket Access	Medium
AUTH	Weak Password Recovery	Critical
SQLI-01	SQL Injection – Authentication Bypass	Critical
SQLI-02	SQL Injection – Product Search and Data Extraction	High
XSS	Reflected Cross-Site Scripting – Search Parameter	Medium
CSRF	Cross-Site Request Forgery – Profile Update	Medium
OR	Open Redirect – Unvalidated Redirect Parameter	Medium

Table 1.1: Summary of confirmed vulnerabilities

Severity Distribution

The confirmed findings consist of three Critical-severity findings, one High-severity finding, and four Medium-severity findings.

- **Critical:** 3 findings
- **High:** 1 finding
- **Medium:** 4 findings
- **Low:** 0 findings

The Critical findings include Broken Access Control, SQL Injection – Authentication Bypass, and Weak Password Recovery. The single High-severity finding is SQL Injection – Product Search and Data Extraction. Reflected XSS, IDOR, CSRF, and Open Redirect were classified as Medium severity based on the demonstrated impact and exploitation conditions.

Key Security Observations

The assessment identified weaknesses across several important application security controls.

The SQL Injection findings demonstrate insufficient protection of user-controlled input used by database-related functionality. The login functionality was found to be susceptible to authentication bypass, while the product search functionality was found to allow unauthorized database interaction under the tested conditions.

The Broken Access Control finding demonstrates a weakness in server-side authorization and privilege assignment. The Weak Password Recovery finding demonstrates that an attacker can take over the affected account without knowledge of the original password. If the same weakness affected a privileged account, it could potentially lead to unauthorized administrative access.

The remaining findings affect resource authorization, client-side script execution, state-changing requests, and redirect handling. Together, these findings demonstrate that security controls should be strengthened across multiple areas of the application.

Business and Security Impact

The confirmed vulnerabilities may affect the confidentiality, integrity, and security of application resources depending on the specific vulnerability and exploitation conditions.

The Critical and High severity findings require particular attention because they affect authentication, authorization, account recovery, and database interaction.

If similar weaknesses existed in a production application handling real user information, successful exploitation could result in unauthorized access, modification of application resources, exposure of data, account compromise, or execution of attacker-controlled content.

Recommended Actions

The identified vulnerabilities should be addressed according to their severity and associated risk.

Priority should be given to implementing parameterized database queries, enforcing server-side authorization and role assignment, strengthening password recovery controls, implementing object-level authorization checks, and applying appropriate output encoding and request protection.

Additional controls should include secure redirect validation, appropriate security headers, least-privilege access, server-side input validation, and security monitoring.

After remediation, all confirmed findings should be retested using the original proof-of-concept techniques and additional relevant test cases to verify that the vulnerabilities have been effectively addressed.

Report Scope

This report documents the testing activities, identified vulnerabilities, supporting evidence, impact assessment, and recommended remediation measures within the defined assessment scope.

The findings and conclusions presented in this report are based on the testing performed against the specified OWASP Juice Shop environment during this engagement.

2 Introduction

This report presents the results of a Vulnerability Assessment and Penetration Testing (VAPT) assessment conducted against a locally hosted OWASP Juice Shop application. OWASP Juice Shop is a deliberately vulnerable web application that provides a controlled environment for identifying and understanding common web application security weaknesses.

The assessment was performed in a controlled environment with the objective of examining the security of the application from an attacker-oriented perspective. The testing process combined automated security tools with manual testing and verification techniques.

The assessment was organized into a structured sequence of activities, beginning with planning and scoping, followed by reconnaissance, vulnerability scanning and assessment, and manual verification and proof-of-concept testing. The results from these activities were used to identify potential security weaknesses and select findings for further manual analysis and validation.

The assessment confirmed eight security vulnerabilities covering multiple security areas, including Broken Access Control, Insecure Direct Object Reference (IDOR), Weak Password Recovery, SQL Injection, Reflected Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and Open Redirect. Evidence was collected during testing to support each confirmed finding and provide a clear record of the assessment activities.

The confirmed findings consisted of three Critical-severity findings, one High-severity finding, and four Medium-severity findings. The Critical findings included Broken Access Control, SQL Injection – Authentication Bypass, and Weak Password Recovery. The single High-severity finding was SQL Injection – Product Search and Data Extraction, while Reflected XSS, IDOR, CSRF, and Open Redirect were classified as Medium severity.

The remainder of this report presents the assessment objectives, scope, methodology, tools, phase-wise testing activities, confirmed vulnerabilities, risk assessment, remediation recommendations, retesting activities, and supporting evidence.

3 Assessment Approach

3.1 Project Objectives

This section defines the objectives of the Vulnerability Assessment and Penetration Testing (VAPT) engagement conducted against the locally hosted OWASP Juice Shop application.

The primary objective of the assessment was to identify and manually validate security vulnerabilities that could affect the confidentiality, integrity, and availability of application resources and user functionality.

The specific objectives of the assessment were:

- **Identify security vulnerabilities** in the OWASP Juice Shop web application through systematic security testing.
- **Perform reconnaissance** to identify accessible endpoints, application functionality, parameters, and other relevant attack surfaces.
- **Perform automated vulnerability assessment** using appropriate security testing tools to identify potential vulnerabilities and security weaknesses.
- **Perform manual security testing** to verify vulnerabilities identified through automated scanning and to identify weaknesses that may not be detected automatically.
- **Validate identified vulnerabilities** using controlled proof-of-concept techniques without causing unnecessary disruption to the application.
- **Assess the security impact** of each confirmed vulnerability based on the demonstrated exploitation and affected application functionality.
- **Collect supporting evidence** such as HTTP requests and responses, scanner results, screenshots, and proof-of-concept results for each confirmed finding.
- **Classify confirmed vulnerabilities** according to their severity and relevant security classification frameworks, including OWASP and CWE where applicable.
- **Provide remediation recommendations** for the confirmed vulnerabilities to help strengthen the application's security controls.
- **Define retesting requirements** to verify that vulnerabilities are effectively addressed after remediation.

The assessment was conducted within the defined scope and controlled testing environment. Testing activities were focused on identifying and validating security weaknesses in the target application rather than causing permanent modification or disruption to the system.

3.2 Scope and Rules of Engagement

This section defines the target, boundaries, and rules followed during the Vulnerability Assessment and Penetration Testing (VAPT) engagement. The assessment was performed in a controlled local laboratory environment.

3.2.1 Assessment Scope

The assessment was performed against a locally hosted instance of the OWASP Juice Shop application. The target and testing boundaries were defined before the assessment began.

Assessment Item	Details
Target Application	OWASP Juice Shop
Target URL	http://127.0.0.1:3000
Environment	Local laboratory environment
Assessment Type	Web Application VAPT
Testing Approach	Manual and automated testing

Table 3.1: Assessment target and environment

3.2.2 In-Scope Components

The following parts of the application were included in the assessment:

- OWASP Juice Shop web application
- Application pages, routes, and endpoints
- REST API endpoints
- Authentication and session mechanisms
- User-controlled input fields and parameters
- Application access-control mechanisms
- HTTP communication between the client and server
- Security-related application configurations

3.2.3 Out-of-Scope Components

The following components and activities were not included in the assessment:

- External websites and third-party services

- Other devices or systems on the local network
- Exploitation of the host operating system
- Denial-of-Service (DoS) testing
- Any system outside the defined OWASP Juice Shop laboratory environment

3.2.4 Rules of Engagement

The following rules were followed during the assessment:

- Testing was restricted to the locally hosted OWASP Juice Shop instance.
- Testing activities were limited to the defined assessment scope.
- No testing was performed against real-world or unauthorized systems.
- Denial-of-Service testing was excluded from the assessment.
- Vulnerabilities identified during testing were documented with appropriate supporting evidence.
- Testing was performed in a controlled laboratory environment for educational and cyber-security training purposes.

3.2.5 Assessment Authorization

The OWASP Juice Shop application was hosted locally as an intentionally vulnerable application for security testing and educational purposes.

All activities documented in this report were therefore limited to the defined local laboratory environment and the specified target application.

3.3 VAPT Methodology

The assessment followed a structured Vulnerability Assessment and Penetration Testing (VAPT) methodology. The methodology was designed to move from initial planning and reconnaissance to vulnerability identification, validation, documentation, and remediation.

Each phase shown above is documented as its own chapter later in this report: Phase 1 in Chapter 4, Phase 2 in Chapter 5, Phase 3 in Chapter 6, Phase 4 in Chapter 7, Phase 5 in Chapter 8, Phase 6 in Chapter 9, and Phase 7 in Chapter 10.

3.4 Tools and Technologies

The assessment used a combination of manual and automated security testing tools. Each tool was selected based on the specific activity it supported during reconnaissance, vulnerability assessment, and manual validation. Detailed usage of each tool, along with evidence, is documented within the relevant phase chapters later in this report.

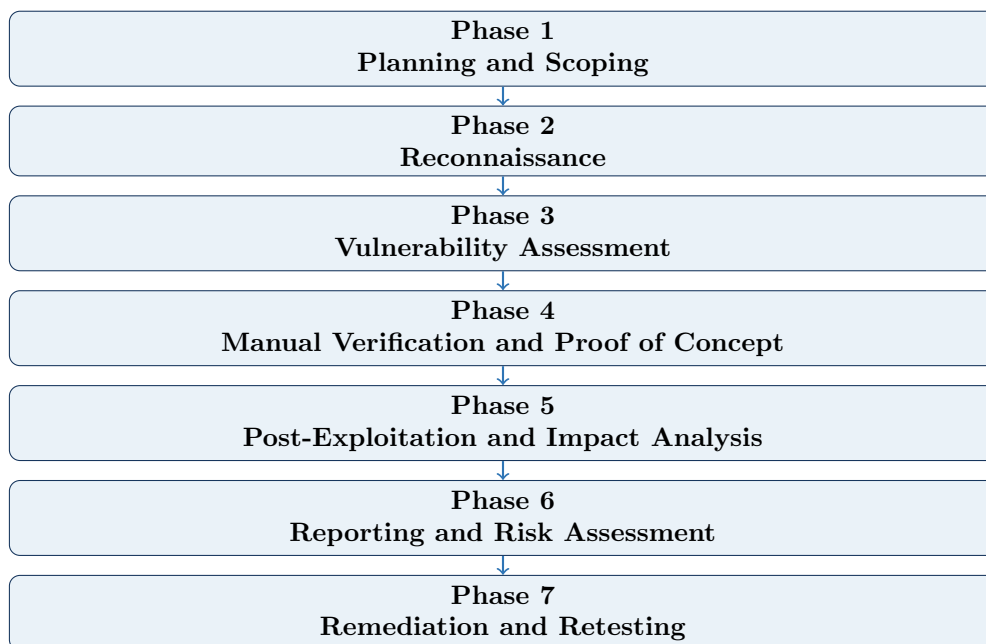


Figure 3.1: VAPT assessment methodology

Tool	Primary Purpose	Assessment Activity
Browser	Manual exploration of application pages, functions, and accessible resources	Reconnaissance
Nmap	Port, service, and basic network discovery	Reconnaissance
Wappalyzer	Identification of technologies, frameworks, and application components	Technology Identification
robots.txt Review	Manual review of the application's robots.txt file to identify disclosed or restricted paths	Reconnaissance
Burp Suite	Interception and analysis of HTTP requests and responses, including headers, cookies, methods, and parameters	Manual Testing and Validation
OWASP ZAP	Application crawling, Spider and AJAX Spider discovery, passive scanning, and active security scanning	Reconnaissance and Vulnerability Assessment
ffuf	Discovery of directories, endpoints, and accessible application paths	Endpoint Discovery
SQLMap	Automated testing and verification of SQL Injection vulnerabilities	Vulnerability Assessment

Table 3.2: Security tools used during the assessment

4 Phase 1 – Planning and Scoping

This phase established the foundation for the OWASP Juice Shop VAPT assessment. The full target definition, scope, and rules of engagement are defined in Chapter 3 (Assessment Approach, Section 3.2) and were confirmed and locked in during this phase before the technical assessment began.

4.1 Phase Objectives

The main purpose of the planning and scoping phase was to establish a clear and controlled assessment environment. The objectives of this phase were to:

- Define the target application and assessment environment.
- Establish the scope of the security assessment.
- Identify the testing approach and assessment boundaries.
- Define the rules of engagement for the assessment.
- Establish the methodology to be followed during the subsequent phases.

4.2 Assessment Methodology

The project was organized into the following seven VAPT phases, as detailed in Chapter 3, Section 3.3:

1. Planning and Scoping
2. Reconnaissance
3. Vulnerability Assessment
4. Manual Verification and Proof of Concept
5. Post-Exploitation and Impact Analysis
6. Reporting and Risk Assessment
7. Remediation and Retesting

The planning phase established the foundation for the reconnaissance, vulnerability assessment, and manual verification activities that followed.

4.3 Expected Deliverables

The planned assessment deliverables included:

- Reconnaissance findings
- Vulnerability scanning and assessment results
- Confirmed vulnerability findings
- Proof-of-concept evidence
- Risk and impact analysis
- Remediation recommendations
- Final VAPT report

4.4 Phase 1 Summary

Table 4.1 summarizes the outcomes confirmed during this phase before the assessment progressed to reconnaissance.

Planning Item	Outcome
Target Definition	Confirmed (Section 3.2)
Assessment Scope	Confirmed (Section 3.2)
Rules of Engagement	Confirmed (Section 3.2)
Assessment Methodology	Confirmed (Section 3.3)
Authorization & Environment	Confirmed – local, isolated laboratory instance

Table 4.1: Phase 1 planning outcomes

4.5 Phase Completion

Phase 1 was completed after the project scope, target, objectives, testing boundaries, and methodology had been confirmed. The assessment then progressed to the reconnaissance phase.

5 Phase 2 – Reconnaissance

The reconnaissance phase focused on understanding the target application, identifying exposed services and technologies, and mapping the observable application attack surface before vulnerability assessment activities began.

5.1 Objective

The objective of this phase was to identify the target application, discover its exposed network and application-level components, identify the technologies in use, and collect information that could support the subsequent vulnerability assessment.

The reconnaissance activities included technology identification, review of accessible application information, port and service discovery, directory and endpoint discovery, HTTP traffic analysis, and automated application crawling. The tools used during this phase are listed in Table 3.2 in Chapter 3, Section 3.4.

5.2 Target Identification

The target application was confirmed to be accessible through the local browser at `http://127.0.0.1:3000`. The application loaded successfully and was available for subsequent reconnaissance and security testing.

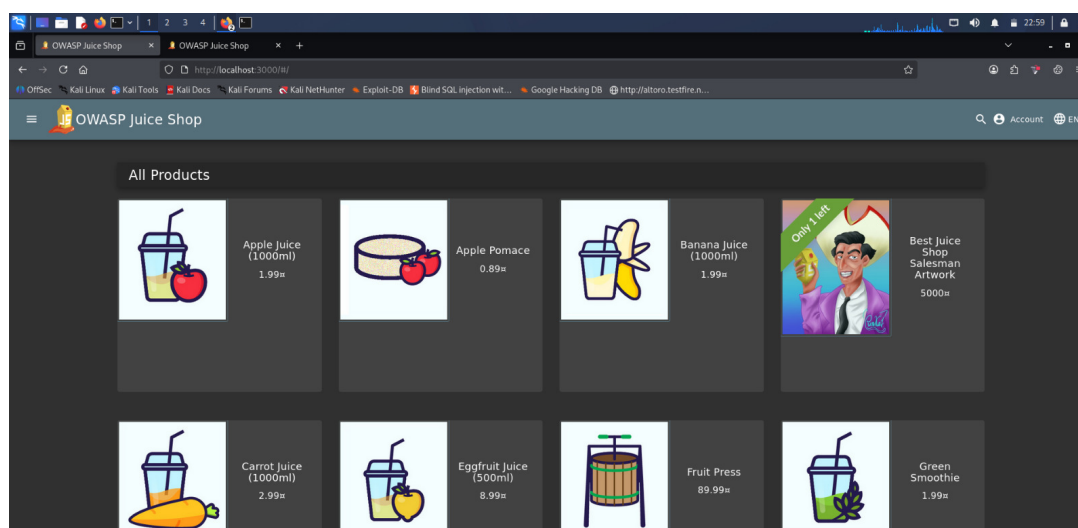


Figure 5.1: OWASP Juice Shop target application accessible at `http://127.0.0.1:3000`.

5.3 Port and Service Discovery

Nmap was used to identify open ports and gather basic service information from the target host. The scan identified port 3000/tcp as open, which was consistent with the locally hosted web application.

```
root@kali: /home/kali # nmap -sT 10.10.10.10
Nmap scan report for 10.10.10.10
Host is up (0.0000000s latency).
Not shown: 65535 closed tcp ports (reset)
PORT: 3000/tcp open
3000/tcp open: http
OS: Linux 3.10.0-1062.el7.x86_64
Nmap done: 1 IP address (1 host up) scanned in 0.44 seconds

root@kali: /home/kali # nmap -sV 10.10.10.10
Nmap scan report for 10.10.10.10
Host is up (0.0000000s latency).
Not shown: 65535 closed tcp ports (reset)
PORT: 3000/tcp open
3000/tcp open: http
OS: Linux 3.10.0-1062.el7.x86_64
Service detected: http
Service version: 1.0
Service fingerprint: 4017021.200 OK
Service detection method: Nmap
Service detection results:
Service: http
Version: 1.0
Fingerprint: 4017021.200 OK
Detection method: Nmap
Service detection results:
Service: http
Version: 1.0
Fingerprint: 4017021.200 OK
Detection method: Nmap
```

Figure 5.2: Nmap port scan results showing port 3000/tcp open.

A service-detection scan was also performed to gather additional information about the service exposed on the identified port.

```
root@kali: /home/kali # nmap -sV 10.10.10.10
Nmap scan report for 10.10.10.10
Host is up (0.0000000s latency).
Not shown: 65535 closed tcp ports (reset)
PORT: 3000/tcp open
3000/tcp open: http
OS: Linux 3.10.0-1062.el7.x86_64
Service detected: http
Service version: 1.0
Service fingerprint: 4017021.200 OK
Service detection method: Nmap
Service detection results:
Service: http
Version: 1.0
Fingerprint: 4017021.200 OK
Detection method: Nmap
Service detection results:
Service: http
Version: 1.0
Fingerprint: 4017021.200 OK
Detection method: Nmap
```

Figure 5.3: Nmap service detection results for the identified web service.

5.4 Technology Identification

Wappalizer was used to identify technologies and frameworks associated with the target application. The results provided information about the client-side frameworks, libraries, content delivery services, and programming technologies observed during reconnaissance.

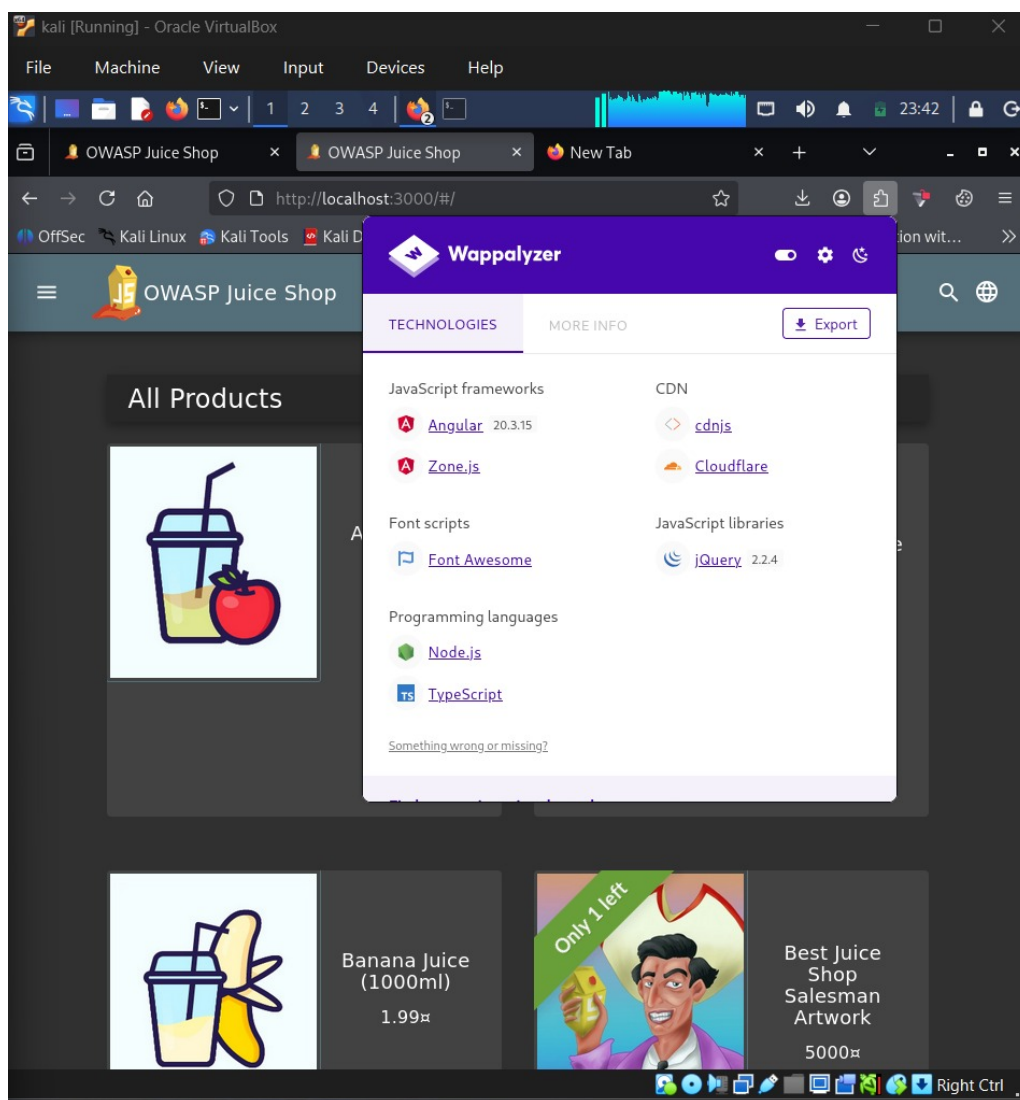


Figure 5.4: Wappalizer technology identification results.

5.5 robots.txt Analysis

The application's `robots.txt` file was directly accessed and reviewed for path information that could reveal additional application resources.

The file contained a disallowed `/ftp` path, providing an additional resource for reconnaissance and subsequent security testing.

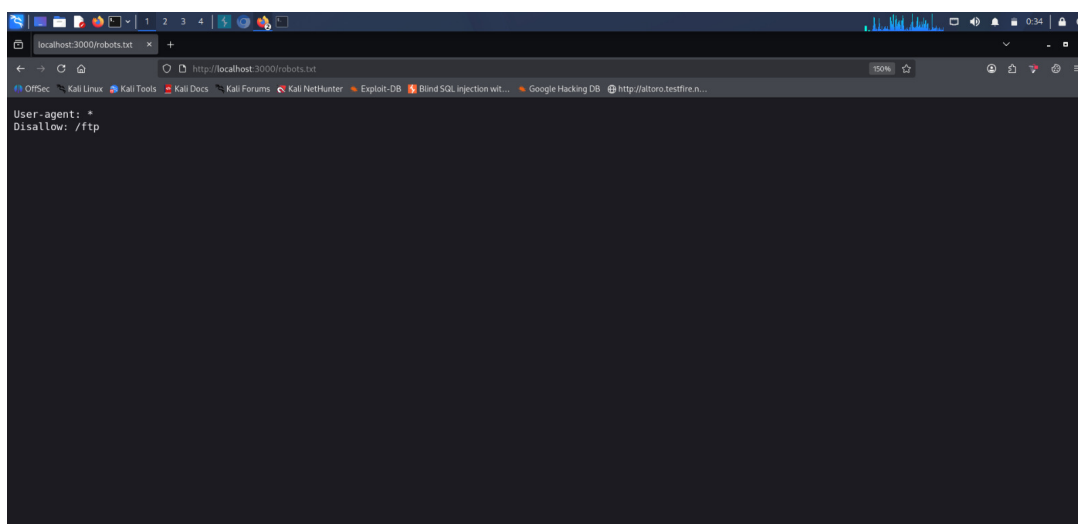


Figure 5.5: Contents of the application's robots.txt file.

5.6 HTTP Request and Response Analysis

Burp Suite was used as an intercepting proxy to observe HTTP communication between the browser and the target application.

The captured traffic provided visibility into the HTTP methods, request headers, cookies, target paths, and server responses. A successful HTTP/1.1 200 OK response was observed for the captured request.

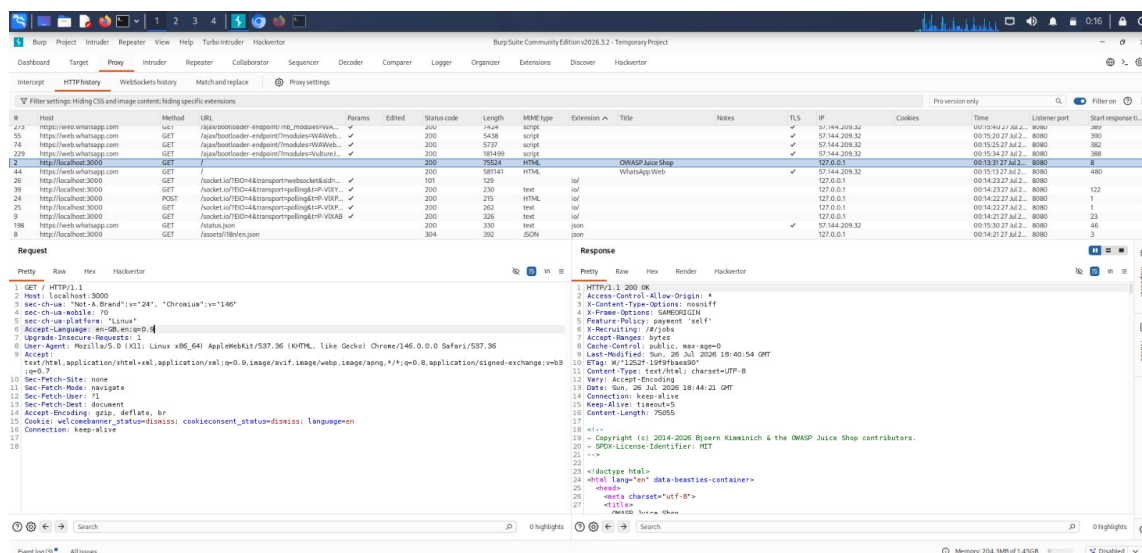


Figure 5.6: Burp Suite HTTP request and response captured during reconnaissance.

5.7 Directory and Endpoint Discovery

ffuf was used to perform active directory and endpoint discovery against the target application. A commonly used web-content wordlist was used to identify accessible paths that were not necessarily visible through normal application navigation.

The screenshot shows a Kali Linux desktop environment. On the left, a terminal window is open, displaying a curl command and its output. The command is:

```
curl -s http://127.0.0.1:3000/FUZZ -u /usr/share/seclists/Discovery/Web-Content/commu.txt -fs 75055
```

The output shows a directory listing of /ftp, including files like quarantine, acquisitions.md, announcement_encrypted.md, coupons_2013.md.bak, easter.e.gg, encrypt.py, and incident-support.kdwx.

On the right, a web browser window is open, displaying the directory listing page. The browser's address bar shows the URL http://localhost:3000/ftp. The page content is a directory listing of /ftp, including files like quarantine, acquisitions.md, announcement_encrypted.md, coupons_2013.md.bak, easter.e.gg, encrypt.py, and incident-support.kdwx.

5.8 OWASP ZAP Spider

The completed Spider scan reported:

- The crawl identified a number of application resources and REST-style endpoints that expanded the observable attack surface.



5.9 OWASP ZAP AJAX Spider

The OWASP ZAP AJAX Spider was additionally used to crawl the application through browser-based navigation and identify resources and routes associated with client-side application behavior.

The completed AJAX Spider scan reported:

- **URLs Crawled: 383**

The AJAX crawl provided additional visibility into dynamically requested resources and JavaScript-driven application behavior.

Processed	ID	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. Body	Highest Alert	Note	Tags
	723	02/08/26, 10:59:29 pm	GET	http://127.0.0.1:3000/	200	OK	7 ms	469 bytes	75,055 bytes			
	724	02/08/26, 10:59:29 pm	GET	http://127.0.0.1:3000/polyfills.js	200	OK	23 ms	481 bytes	34,844 bytes			
	725	02/08/26, 10:59:29 pm	GET	http://127.0.0.1:3000/runtime.js	200	OK	19 ms	479 bytes	3,339 bytes			
	726	02/08/26, 10:59:29 pm	GET	http://127.0.0.1:3000/main.js	200	OK	25 ms	483 bytes	450,237 bytes			
	727	02/08/26, 10:59:29 pm	GET	http://127.0.0.1:3000/vendor.js	200	OK	48 ms	485 bytes	1,692,408 bytes			
	728	02/08/26, 10:59:29 pm	GET	http://cdnjs.cloudflare.com/ajax/libs/cookieconsent...	200	OK	627 ms	923 bytes	20,808 bytes			
	729	02/08/26, 10:59:29 pm	GET	http://cdnjs.cloudflare.com/ajax/libs/cookieconsent...	200	OK	602 ms	906 bytes	4,064 bytes			
	730	02/08/26, 10:59:29 pm	GET	http://cdnjs.cloudflare.com/ajax/libs/jquery/2.2.4...	200	OK	695 ms	927 bytes	85,578 bytes			
	731	02/08/26, 10:59:30 pm	GET	http://127.0.0.1:3000/styles.css	200	OK	9 ms	469 bytes	640,267 bytes			
	732	02/08/26, 10:59:30 pm	GET	http://127.0.0.1:3000/assets/18n/en.json	200	OK	15 ms	475 bytes	33,094 bytes			
	733	02/08/26, 10:59:30 pm	GET	http://127.0.0.1:3000/socket.io/?EIO=4&transp...	200	OK	11 ms	230 bytes	96 bytes			
	734	02/08/26, 10:59:30 pm	GET	http://127.0.0.1:3000/rest/admin/application-ve...	200	OK	7 ms	384 bytes	20 bytes			
	735	02/08/26, 10:59:30 pm	GET	http://127.0.0.1:3000/rest/admin/application-co...	200	OK	5 ms	389 bytes	21,730 bytes			
	736	02/08/26, 10:59:30 pm	GET	http://127.0.0.1:3000/rest/admin/application-co...	304	Not Modified	8 ms	304 bytes	0 bytes			
	737	02/08/26, 10:59:30 pm	GET	http://127.0.0.1:3000/rest/admin/application-co...	200	OK	54 ms	389 bytes	21,730 bytes			
	738	02/08/26, 10:59:30 pm	GET	http://127.0.0.1:3000/rest/admin/application-co...	200	OK	37 ms	300 bytes	21,730 bytes			

Figure 5.9: OWASP ZAP AJAX Spider results showing 383 URLs crawled.

5.10 Reconnaissance Summary

The reconnaissance activities established an initial view of the application's observable attack surface.

The target was confirmed to be accessible at `http://127.0.0.1:3000`, and Nmap identified port `3000/tcp` as open. Wappalyzer identified multiple technologies and frameworks associated with the application.

The review of `robots.txt` revealed the `/ftp` path, while ffuf identified additional application paths and resources. Burp Suite provided visibility into HTTP request and response behavior. OWASP ZAP further expanded the observable application map through both traditional and AJAX-based crawling.

The information collected during this phase was used to guide vulnerability scanning and assessment activities in Phase 3 and manual verification and proof-of-concept testing in Phase 4.

Metric	Result
Target	http://127.0.0.1:3000
Open Port Identified	3000/tcp
ffuf Discovery	Multiple accessible paths identified
ZAP Spider	398 URLs / 255 nodes
ZAP AJAX Spider	383 URLs crawled
robots.txt Observation	/ftp path disclosed
HTTP Analysis	Successful HTTP/1.1 200 OK response observed

Table 5.1: Reconnaissance phase summary

5.11 Phase 3 Preparation

The reconnaissance results established the initial attack surface that was used to guide Phase 3 vulnerability scanning and assessment activities.

A complete de-duplicated endpoint inventory is not included in this section until the raw ffuf, ZAP Spider, and AJAX Spider outputs have been consolidated. This avoids presenting a partial endpoint list as a complete inventory.

6 Phase 3 – Vulnerability Assessment

6.1 Objective

Phase 3 was performed to identify possible security weaknesses in the OWASP Juice Shop application using automated scanning.

The main tool used in this phase was OWASP ZAP. The scan was used to find potential vulnerabilities that required further testing.

The alerts generated during this phase were treated as potential findings. They were not considered confirmed vulnerabilities until they were manually tested in Phase 4.

6.2 Scanning Methodology

The following scanning activities were performed using OWASP ZAP:

1. **Spider Scan** – The traditional Spider was used to crawl the application and discover available pages, links, and endpoints.
2. **AJAX Spider** – The AJAX Spider was used because Juice Shop is a JavaScript-based application. It helped discover additional pages and content that may not be found by the traditional Spider.
3. **Passive Scan** – ZAP analyzed the traffic generated during browsing and crawling. This helped identify common security issues without actively attacking the application.
4. **Active Scan** – ZAP actively tested the discovered application endpoints for possible vulnerabilities, including SQL Injection, Cross-Site Scripting, and other web security issues.
5. **Alert Review** – The alerts generated by ZAP were reviewed and compared with the application's actual behavior. Findings that required further testing were carried forward to Phase 4 for manual validation.

6.3 ZAP Scan Evidence

The following evidence shows the OWASP ZAP alerts identified during the scanning process for the two findings that were flagged by automated scanning: SQL Injection (SQLI-02) and Open Redirect (OR).

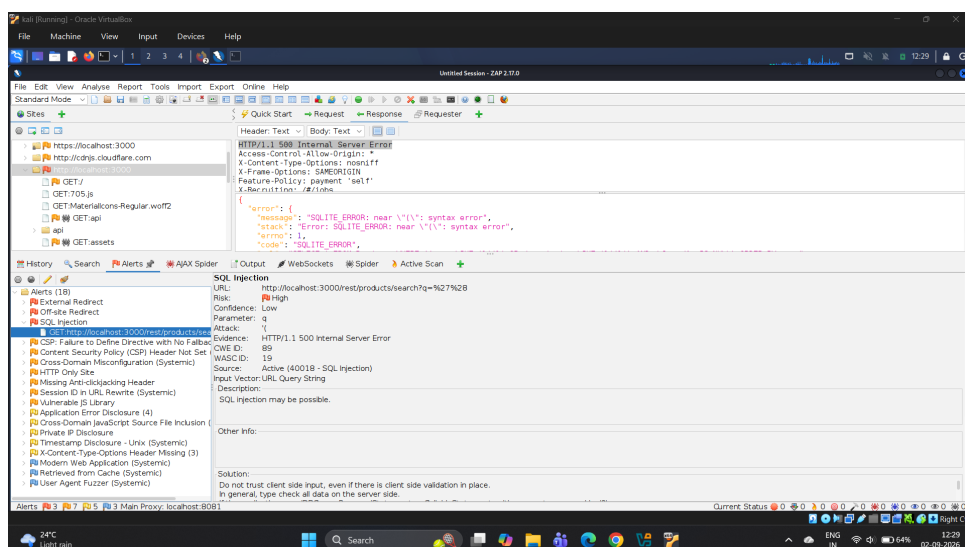


Figure 6.1: OWASP ZAP identifying a possible SQL Injection vulnerability in the product search parameter

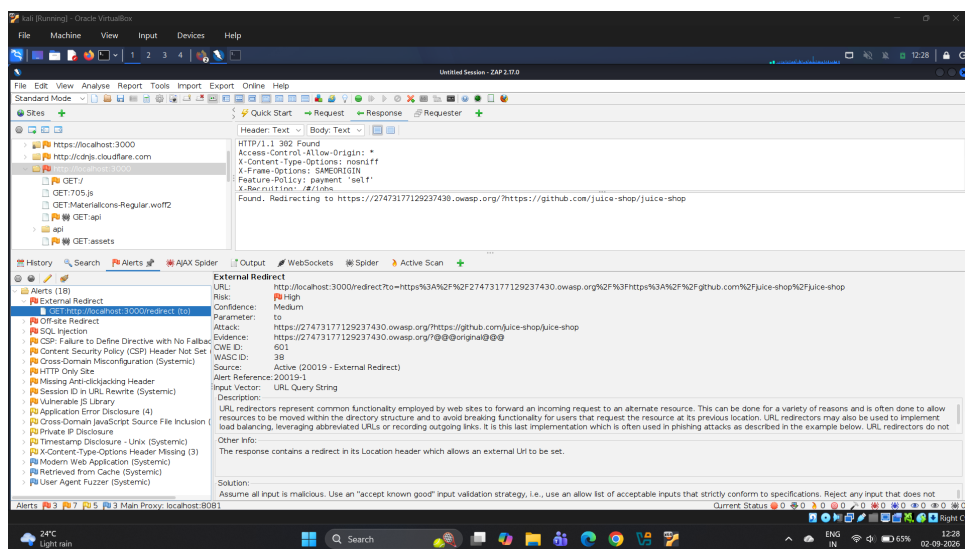


Figure 6.2: OWASP ZAP identifying a possible Open Redirect vulnerability in the redirect parameter

Both alerts shown above were treated as potential vulnerabilities. Further testing was performed in Phase 4 using Burp Suite (and SQLMap for SQLI-02) to confirm each issue and determine its actual impact. Full exploitation details are provided in Sections 7.4 and 7.7.

6.4 Scan Results

The OWASP ZAP scan generated a total of 18 alerts.

The alerts were reviewed before deciding which issues required detailed manual testing. An alert from an automated scanner does not always mean that a real vulnerability exists. Therefore, the findings were manually tested before being included as confirmed vulnerabilities in this report.

Scan Result	Count
Total ZAP Alerts	18
Findings Selected for Phase 4 Validation	2
Other Alerts Not Carried Forward	16

Table 6.1: Summary of OWASP ZAP scan results

6.5 Findings Selected for Manual Validation

The following two findings were identified through automated scanning and selected for further testing and evidence collection in Phase 4:

Finding ID	Vulnerability
SQLI-02	SQL Injection – Product Search Parameter
OR	Open Redirect – Unvalidated Redirect Parameter

Table 6.2: Findings selected for Phase 4 validation (ZAP-detected)

The remaining confirmed findings – BAC, IDOR, AUTH, SQLI-01 (Authentication Bypass through Login), XSS, and CSRF – were not flagged by the OWASP ZAP automated scan and were instead identified through manual testing during Phase 4, as documented in their respective finding sections.

6.6 Phase 3 to Phase 4

Phase 3 was mainly used for automated discovery. The alerts provided an initial indication of possible security weaknesses for two of the eight confirmed findings.

In Phase 4, both the ZAP-flagged findings and the additional manually-identified findings were tested and confirmed using Burp Suite and other appropriate tools. Evidence was collected to prove whether each issue was actually exploitable and to understand its security impact.

Therefore, the report follows this process:

**Automated Scanning / Manual Discovery → Alert Review → Manual Validation
→ Evidence Collection → Confirmed Finding**

6.7 Note

The detailed technical information and exploitation evidence for the confirmed vulnerabilities are provided in the Phase 4 finding chapters.

The purpose of Phase 3 is to document how potential vulnerabilities were discovered, while Phase 4 provides the detailed proof and impact of the confirmed vulnerabilities.

7 Phase 4 – Manual Verification and Proof of Concept

7.1 Broken Access Control

Broken Access Control – Privilege Escalation via Registration (BAC)

Finding ID	BAC
Severity	Critical
CVSS 3.1 Score	9.8 (Critical)
CVSS Vector	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Status	Confirmed
OWASP Category	A01:2021 – Broken Access Control
CWE	CWE-269 – Improper Privilege Management
Affected Endpoint	POST /api/Users/
Affected Parameter	role

Table 7.1: Broken Access Control – Privilege Escalation finding details

Description

The user registration function was found to accept a `role` field directly from the client without server-side restriction.

By intercepting the registration request and modifying this field, a normal user was able to register a new account with administrator privileges. The server did not validate whether the client was authorized to set this value, allowing any unauthenticated visitor to create an admin-level account.

Methodology

The registration functionality was tested by creating a normal user account while intercepting the registration request using Burp Suite. The request body was reviewed to identify client-controlled parameters that could affect account privileges. The `role` parameter was then modified from its normal value to `admin` and the modified request was forwarded to the server.

The resulting account was subsequently used to verify whether administrator-level access

had been granted. Successful access to the Administration page confirmed that the server accepted the client-supplied role without performing adequate server-side authorization checks.

Steps to Reproduce

1. Navigate to the registration page and begin creating a new account normally.
 2. Intercept the registration request using Burp Suite before it reaches the server.
 3. Modify the request body to include the following field:
- ```
"role": "admin"
```
4. Forward the modified request to the server.
  5. Log in using the newly created account (User14@gmail.com).
  6. Verify that the account can access the Administration page and its associated functionality.

## Evidence

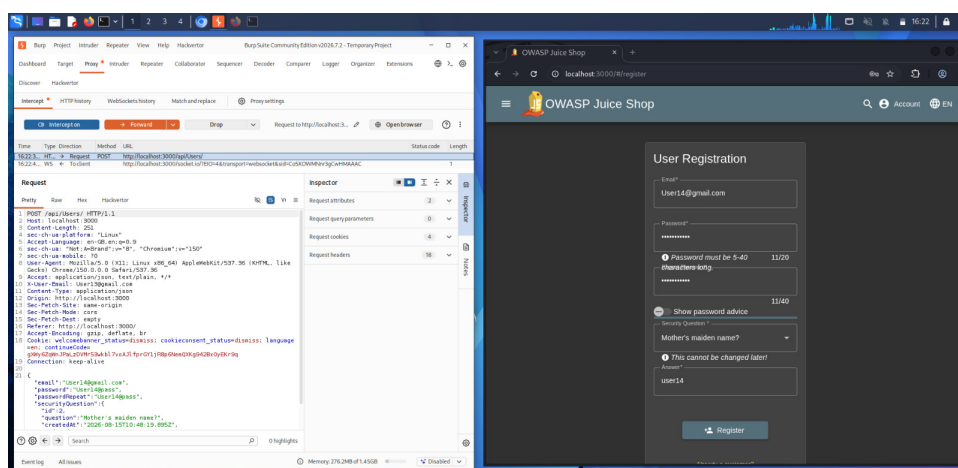


Figure 7.1: Registration request captured in Burp Suite

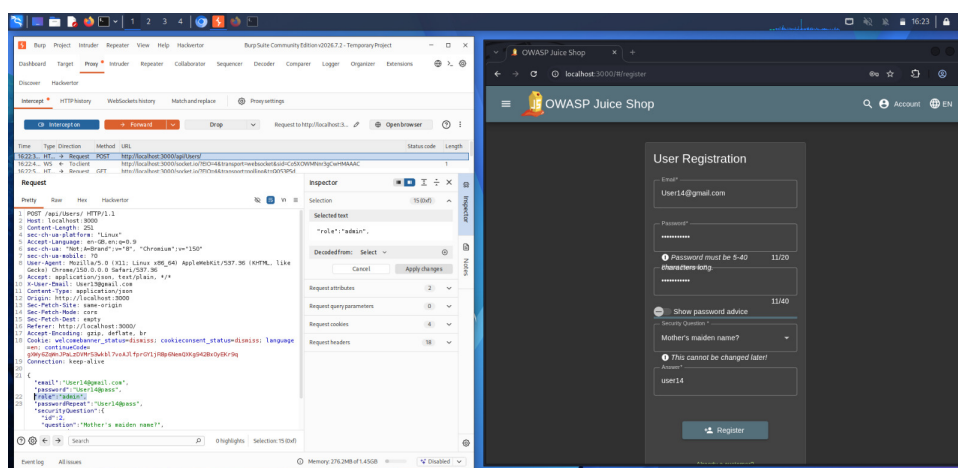


Figure 7.2: Registration request modified to set the role to admin

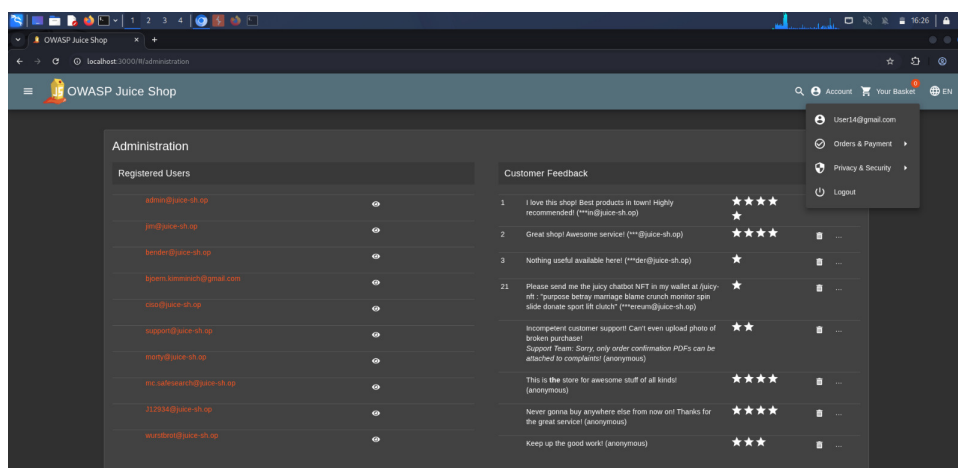


Figure 7.3: Newly created account successfully accessing the Administration page

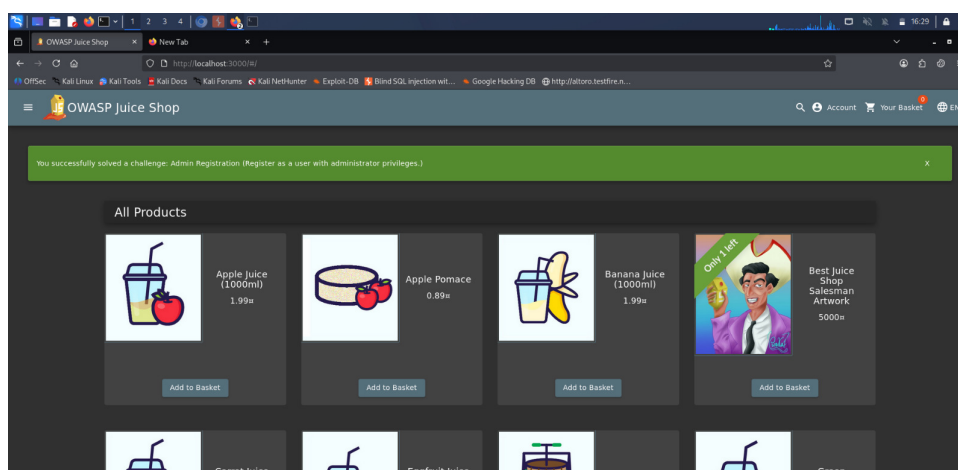


Figure 7.4: Successful completion of the Admin Registration challenge

## Impact

An attacker can create an administrator account without authorization. After logging in with the new account, the attacker can access administrator functions that should only be available to authorized administrators.

This can allow unauthorized management of users, products, and other administrative data.

## Remediation

- Do not allow users to choose their role during registration.
- Always assign the normal user role on the server side when creating a new account.
- Ignore or reject any client-supplied `role` value during normal registration.
- Allow only authorized administrators to change a user's role.
- Perform server-side authorization checks before allowing access to administrator functions.

## Conclusion

The Broken Access Control vulnerability was successfully confirmed by modifying the registration request and assigning the `admin` role to a newly created account.

The account was then able to access the Administration page, confirming that the application does not properly enforce server-side privilege controls during registration.

The finding is therefore classified as a confirmed Critical-severity security issue.

## 7.2 Insecure Direct Object Reference

### Insecure Direct Object Reference – Unauthorized Basket Access (IDOR)

|                           |                                                            |
|---------------------------|------------------------------------------------------------|
| <b>Finding ID</b>         | IDOR                                                       |
| <b>Severity</b>           | Medium                                                     |
| <b>CVSS 3.1 Score</b>     | 6.5 (Medium)                                               |
| <b>CVSS Vector</b>        | AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N                        |
| <b>Status</b>             | Confirmed                                                  |
| <b>OWASP Category</b>     | A01:2021 – Broken Access Control                           |
| <b>CWE</b>                | CWE-639 – Authorization Bypass Through User-Controlled Key |
| <b>Affected Endpoint</b>  | GET /rest/basket/{id}                                      |
| <b>Affected Parameter</b> | id (basket ID in URL path)                                 |

**Table 7.2:** Insecure Direct Object Reference finding details

### Description

The basket endpoint uses the basket ID provided in the URL to return basket information. However, the application does not properly check whether the requested basket belongs to the logged-in user.

During testing, the basket ID was changed from one value to another while using the same authenticated session. The application returned basket information for a different user instead of denying access.

This is an Insecure Direct Object Reference (IDOR) vulnerability. It occurs when an application uses a user-controlled object identifier without properly verifying whether the authenticated user is authorized to access that object.

### Methodology

The basket functionality was tested using Burp Suite. A normal basket request was first captured and the basket ID in the URL was identified. The ID was then changed to another value while keeping the same authenticated session.

The response was examined to determine whether the application properly verified ownership of the requested basket.

## Steps to Reproduce

1. Log in as a normal user and open the shopping basket.
2. Capture the basket request using Burp Suite.
3. Observe the basket ID in the request, for example:

```
GET /rest/basket/7
```

4. Send the request and confirm that the application returns the basket associated with the current user.
5. Change the basket ID to another value, for example:

```
GET /rest/basket/5
```

6. Send the modified request using the same authenticated session.
7. Observe that the application returns the contents of another basket instead of returning an access-denied response.

## Evidence

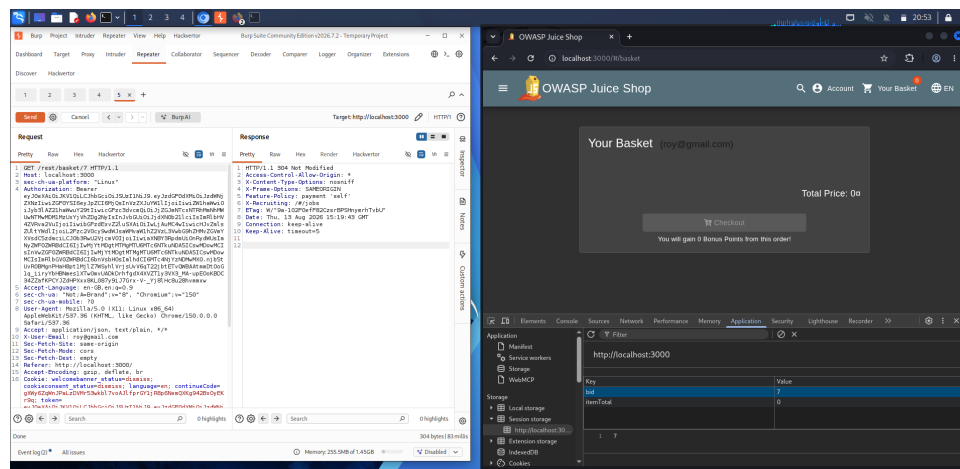


Figure 7.5: Original basket request using the basket ID associated with the logged-in user

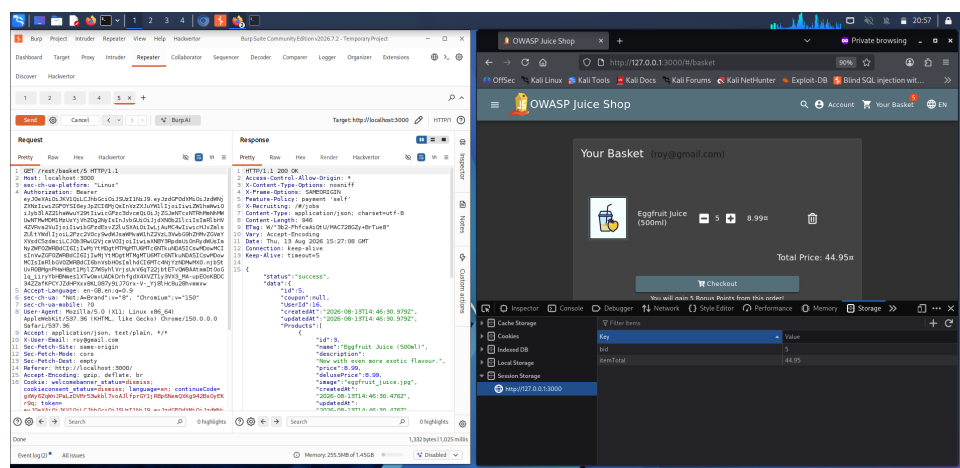


Figure 7.6: Modified basket ID returning another user's basket data

## Impact

A logged-in user can access another user's basket by changing the basket ID in the request.

This can expose information about another user's shopping activity, including the products and quantities stored in the basket.

If basket identifiers are predictable, an attacker may also be able to test additional identifiers and access other baskets when the same authorization weakness exists.

## Remediation

- Verify on the server side that the requested basket belongs to the currently authenticated user.
- Deny access when the user does not have permission to access the requested basket.
- Do not rely solely on the basket ID supplied by the client to determine authorization.
- Apply the same server-side ownership checks to other user-specific resources such as orders, profiles, and payment information.
- Use unpredictable object identifiers where appropriate as an additional security measure. This must not replace proper authorization checks.

## Conclusion

The IDOR vulnerability was successfully confirmed by modifying the basket ID in an authenticated request.

The application returned another user's basket data without verifying whether the logged-in user was authorized to access the requested basket.

The finding is therefore classified as a confirmed Medium-severity Broken Access Control vulnerability.

## 7.3 Authentication Vulnerabilities

## Weak Password Recovery Mechanism via Security Question (AUTH)

|                               |                                                                   |
|-------------------------------|-------------------------------------------------------------------|
| <b>Finding ID</b>             | AUTH                                                              |
| <b>Severity</b>               | Critical                                                          |
| <b>CVSS 3.1 Score</b>         | 9.1 (Critical)                                                    |
| <b>CVSS Vector</b>            | AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N                               |
| <b>Status</b>                 | Confirmed                                                         |
| <b>OWASP Category</b>         | A07:2021 – Identification and Authentication Failures             |
| <b>CWE</b>                    | CWE-640 – Weak Password Recovery Mechanism for Forgotten Password |
| <b>Affected Functionality</b> | Forgot Password / Password Recovery                               |
| <b>Affected Account</b>       | jim@juice-sh.op                                                   |

**Table 7.3:** Weak Password Recovery Mechanism finding details

### Description

The password recovery functionality was found to rely on a security question whose answer could be discovered or guessed using publicly available information.

The application allows a user who provides the correct answer to the security question to set a new password for the target account.

During testing, the security question for the account `jim@juice-sh.op` was:

Your eldest siblings middle name?

The answer was identified as **Samuel** based on publicly available information associated with the account's identity.

After submitting the correct answer, the application allowed a new password to be set successfully.

This demonstrates that the password recovery mechanism does not provide sufficiently strong proof of account ownership.

### Methodology

The Forgot Password functionality was tested manually.

The password recovery process was initiated for the target account. The security question displayed by the application was reviewed.

The answer was then identified using publicly available information related to the target account.

The identified answer was submitted to the password recovery form. The application accepted the answer and allowed a new password to be created.

The successful password reset was used as evidence that the recovery mechanism could be bypassed without knowledge of the original password.

## Steps to Reproduce

1. Open the Forgot Password functionality:

```
http://127.0.0.1:3000/#/forgot-password
```

2. Enter the target email address:

```
jim@juice-sh.op
```

3. Observe the security question:

```
Your eldest siblings middle name?
```

4. Identify the answer using publicly available information associated with the target account.

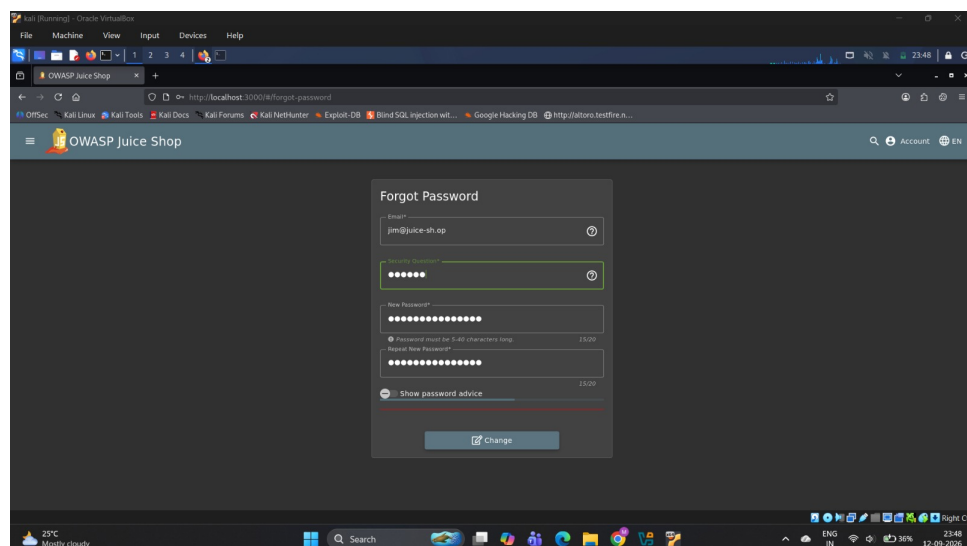
5. Enter the answer:

```
Samuel
```

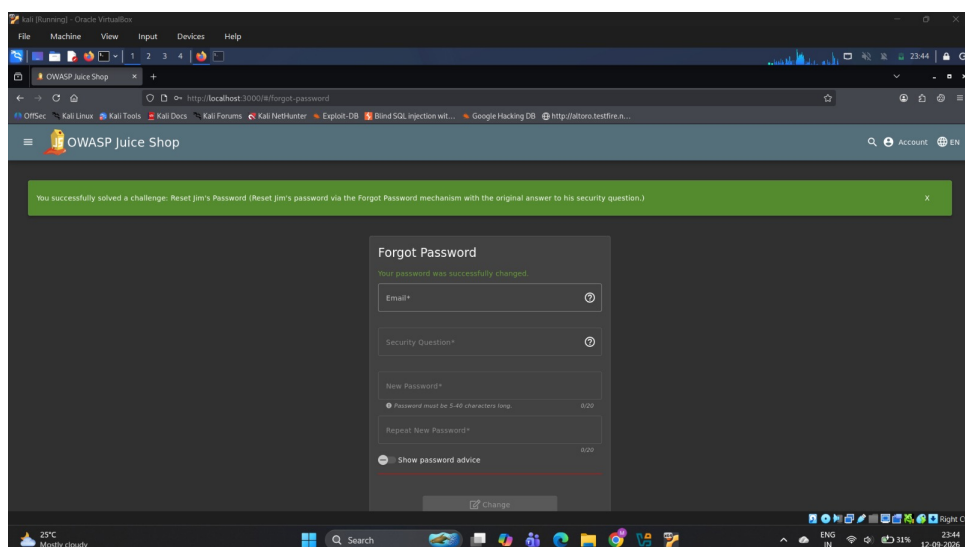
6. Enter a new password and confirm the new password.
7. Submit the password reset form.
8. Observe that the application confirms that the password was successfully changed.

## Evidence

The following evidence demonstrates the weak security-question mechanism and successful password reset.



**Figure 7.7:** Forgot Password process with the security question answered



**Figure 7.8:** Application confirming that the password was successfully changed

## Impact

An attacker who can discover or guess the answer to the security question may be able to reset the victim's password without knowing the original password.

This can result in unauthorized access to the affected account and may lead to account takeover.

The impact may be greater if the affected account has access to sensitive information or privileged functionality.

## Remediation

- Do not rely on easily discoverable security questions as the primary mechanism for password recovery.
- Use a secure password-reset mechanism based on a cryptographically random, single-use, time-limited reset token.
- Send password-reset links or verification codes through a previously verified communication channel.
- Expire password-reset tokens after a short period and invalidate them after successful use.
- Apply rate limiting and monitoring to password-recovery attempts.
- Do not reveal sensitive information that helps an attacker determine whether a target account exists.

## Conclusion

The weak password recovery mechanism was successfully confirmed through manual testing.

The application accepted an answer to a predictable security question and allowed a new password to be set for the target account without requiring the original password.

The finding is therefore classified as a confirmed Critical-severity Identification and Authentication Failure.

## 7.4 SQL Injection

### SQL Injection – Authentication Bypass (Login Endpoint) (SQLI-01)

|                    |                                                                                             |
|--------------------|---------------------------------------------------------------------------------------------|
| Finding ID         | SQLI-01                                                                                     |
| Severity           | Critical                                                                                    |
| CVSS 3.1 Score     | 9.8 (Critical)                                                                              |
| CVSS Vector        | AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H                                                         |
| Status             | Confirmed                                                                                   |
| OWASP Category     | A03:2021 – Injection                                                                        |
| CWE                | CWE-89 – Improper Neutralization of Special Elements used in an SQL Command (SQL Injection) |
| Affected Endpoint  | POST /rest/user/login                                                                       |
| Affected Parameter | email                                                                                       |

Table 7.4: SQL Injection – Authentication Bypass finding details

#### Description

The login function was found to be vulnerable to SQL Injection through the **email** parameter.

A SQL Injection payload was entered into the login form. The application accepted the input and allowed authentication without using valid administrator credentials.

This shows that user input is being used unsafely in the login process. An attacker can use this weakness to bypass the login controls.

#### Methodology

The login functionality was tested manually using the application and Burp Suite.

A SQL Injection payload was entered into the **email** parameter while an arbitrary password was supplied. The resulting HTTP request and server response were then inspected using Burp Suite.

Successful authentication and access to the administrator session were used as the primary evidence for confirming the vulnerability.

#### Steps to Reproduce

1. Navigate to `http://127.0.0.1:3000/#/login`.
2. Enter the following payload in the **email** field:

```
user' or 1=1--
```

3. Enter any value in the password field.

4. Submit the login form.
5. Capture the request using Burp Suite and verify that the request is sent to `POST /rest/user/login`.
6. Observe the server response. The response returns `HTTP/1.1 200 OK` and an authentication token.
7. Verify that the application logs in as the administrator account.

## Evidence

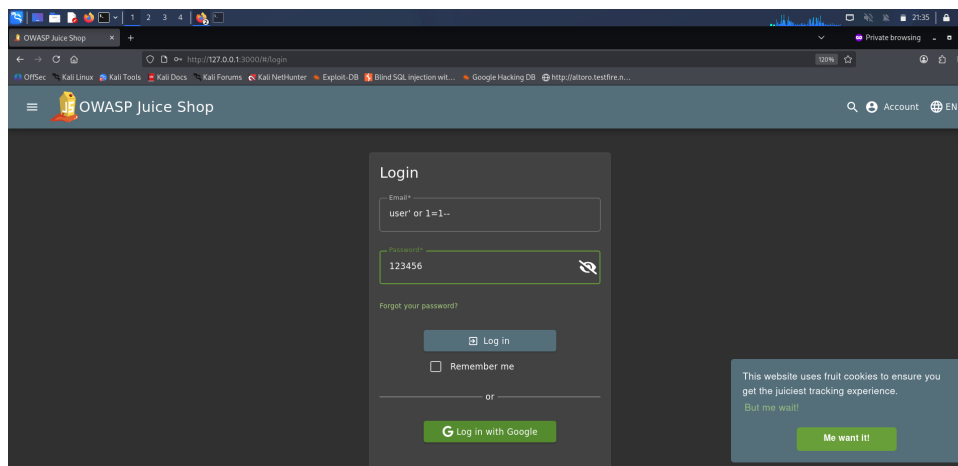


Figure 7.9: SQL Injection payload entered into the login form

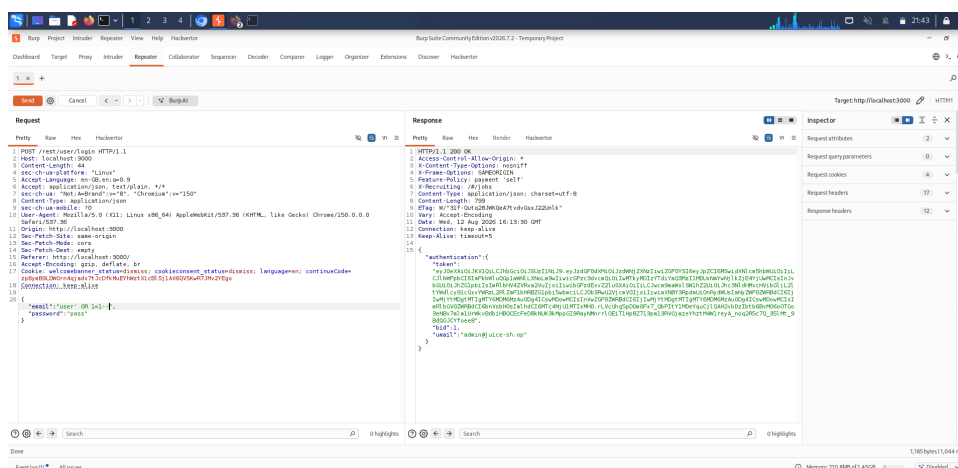


Figure 7.10: Burp Suite request and response showing the injected input

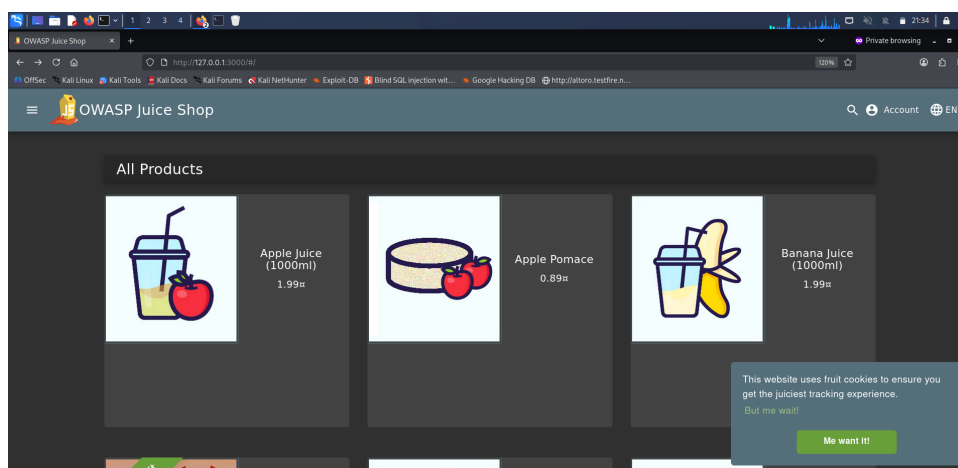


Figure 7.11: Successful login as the administrator account

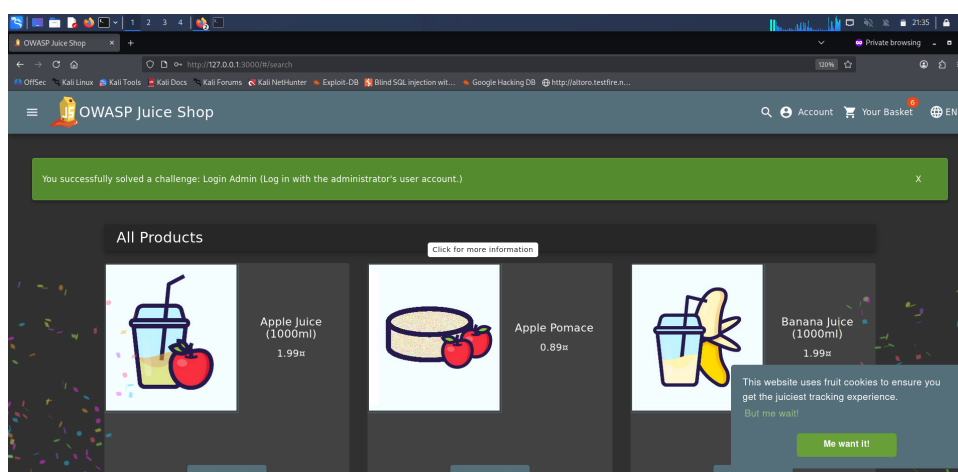


Figure 7.12: Authenticated administrator session after successful exploitation

## Impact

An attacker can bypass the login process without knowing the administrator password.

This can give the attacker access to the administrator account and its available functions. The vulnerability can therefore lead to full account compromise.

## Remediation

- Use parameterized queries or prepared statements for the login database query.
- Never build SQL queries by directly joining user input with SQL statements.
- Validate the `email` parameter on the server side as an additional security measure.
- Use a database account with only the permissions required by the application.
- Add authentication monitoring and rate limiting to reduce repeated attack attempts.

## Conclusion

The SQL Injection vulnerability in the login endpoint was successfully confirmed by injecting SQL syntax into the `email` parameter.

The application authenticated the attacker without requiring valid administrator credentials, demonstrating that the login controls can be bypassed through SQL Injection.

The finding is therefore classified as a confirmed Critical-severity SQL Injection vulnerability.

## SQL Injection – Product Search Parameter (SQLI-02)

|                           |                                                                                             |
|---------------------------|---------------------------------------------------------------------------------------------|
| <b>Finding ID</b>         | SQLI-02                                                                                     |
| <b>Severity</b>           | High                                                                                        |
| <b>CVSS 3.1 Score</b>     | 7.5 (High)                                                                                  |
| <b>CVSS Vector</b>        | AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N                                                         |
| <b>Status</b>             | Confirmed                                                                                   |
| <b>OWASP Category</b>     | A03:2021 – Injection                                                                        |
| <b>CWE</b>                | CWE-89 – Improper Neutralization of Special Elements used in an SQL Command (SQL Injection) |
| <b>Affected Endpoint</b>  | GET /rest/products/search                                                                   |
| <b>Affected Parameter</b> | q                                                                                           |

**Table 7.5:** SQL Injection – Product Search finding details

## Description

The product search function was found to be vulnerable to SQL Injection through the `q` parameter.

The parameter accepts user-controlled input and the input affects the database query. An attacker can send specially crafted input to modify the query and access database information.

Unlike SQLI-01, this vulnerability can be reached without logging in to the application.

## ZAP Detection – Phase 3

During the Phase 3 automated scan, OWASP ZAP reported a possible SQL Injection vulnerability in the `q` parameter of the product search endpoint.

The ZAP alert was treated as a potential finding and was not considered confirmed at this stage. The alert was later tested manually in Phase 4.

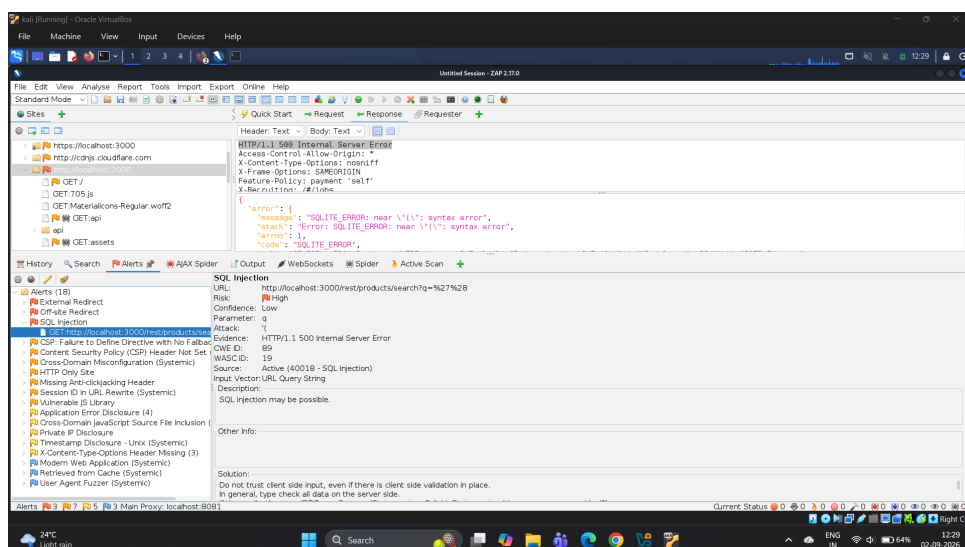


Figure 7.13: OWASP ZAP alert reporting possible SQL Injection in the product search parameter

## Manual Validation – Phase 4

The ZAP finding was manually tested using Burp Suite Repeater.

A crafted input was sent through the `q` parameter and the server response was inspected.

The response showed that the input changed the normal database query behavior. This confirmed that the `q` parameter was vulnerable to SQL Injection.

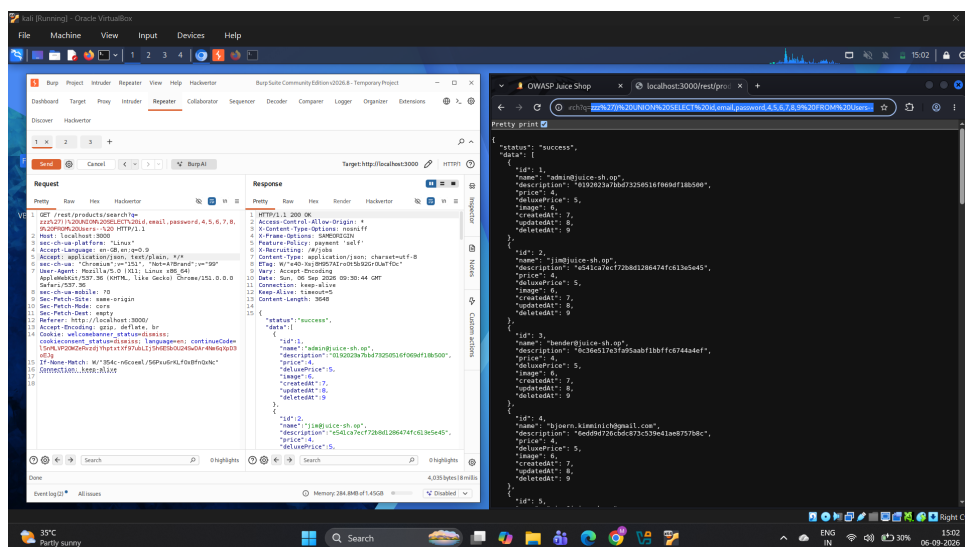


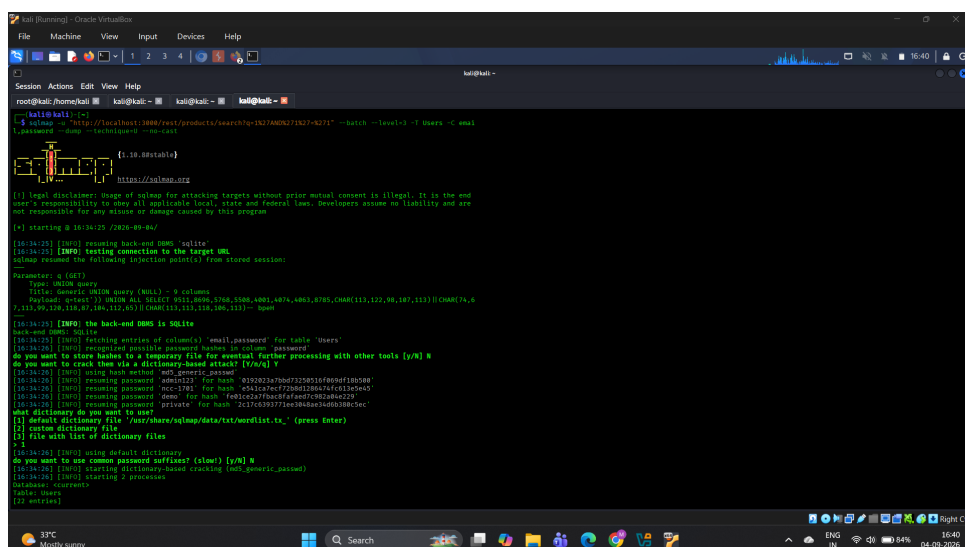
Figure 7.14: Manual confirmation of SQL Injection using Burp Suite Repeater

## Automated Validation – SQLMap

SQLMap was then used to independently confirm the injection point.

SQLMap identified the `q` parameter as injectable and identified SQLite as the database management system.

It also identified the columns of the `Users` table, including `email`, `password`, and `username`.



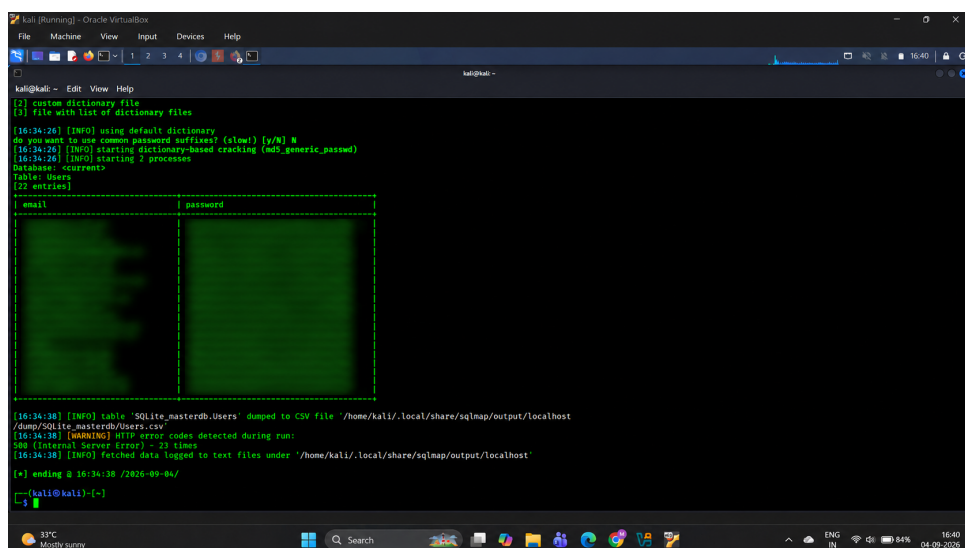
**Figure 7.15:** SQLMap confirming the injection point and identifying the Users table columns

## Data Extraction

To understand the impact of the vulnerability, SQLMap was used to extract data from the `Users` table.

The test returned 22 user records, including email addresses and password hashes. The administrator account was also present in the returned data.

Sensitive credentials and password hashes are not reproduced in this report.



**Figure 7.16:** SQLMap extracting user records from the Users table

## Impact

An unauthenticated attacker can use this vulnerability to access data stored in the application's database.

During testing, user email addresses and password hashes were successfully retrieved from the **Users** table.

If exposed password hashes are weak or reused elsewhere, they may increase the risk of account compromise.

Because the vulnerable endpoint can be accessed without logging in, an attacker does not need an existing account to attempt this attack.

## Remediation

- Use parameterized queries or prepared statements for the product search query.
- Do not directly place the `q` parameter into SQL statements.
- Validate the `q` parameter on the server side as an additional security measure.
- Use a database account with the minimum permissions required by the application.
- Store passwords using a strong password hashing algorithm such as Argon2 or bcrypt with unique salts.

## Conclusion

The SQL Injection vulnerability in the product search endpoint was successfully confirmed through both manual testing with Burp Suite and automated validation using SQLMap.

The vulnerable endpoint was accessible without authentication, and testing demonstrated that database information could be retrieved from the `Users` table.

The finding is therefore classified as a confirmed High-severity SQL Injection vulnerability.

## 7.5 Cross-Site Scripting

### Reflected Cross-Site Scripting – Search Parameter (XSS)

|                           |                                                                      |
|---------------------------|----------------------------------------------------------------------|
| <b>Finding ID</b>         | XSS                                                                  |
| <b>Severity</b>           | Medium                                                               |
| <b>CVSS 3.1 Score</b>     | 6.1 (Medium)                                                         |
| <b>CVSS Vector</b>        | AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N                                  |
| <b>Status</b>             | Confirmed                                                            |
| <b>OWASP Category</b>     | A03:2021 – Injection                                                 |
| <b>CWE</b>                | CWE-79 – Improper Neutralization of Input During Web Page Generation |
| <b>Affected Endpoint</b>  | GET <code>/rest/products/search</code>                               |
| <b>Affected Parameter</b> | <code>q</code>                                                       |

**Table 7.6:** Reflected XSS finding details

## Description

The product search feature was found to be vulnerable to Reflected Cross-Site Scripting (XSS) through the `q` parameter.

The application takes the value from the search parameter and displays it back on the page without properly treating it as plain text. Because of this, an attacker can create a specially crafted search link containing JavaScript.

When a victim opens the malicious link, the injected JavaScript can run in the victim's browser.

The malicious input is not stored in the application. It is reflected back as part of the response, which is why this is called **Reflected XSS**.

## Methodology

The normal product search request was first captured using Burp Suite. The `q` parameter was then tested with different XSS payloads using Burp Intruder.

The responses were checked to identify a payload that was reflected by the application. The successful payload was then opened in the browser. A JavaScript alert appeared, confirming that the injected script was executed.

## Steps to Reproduce

1. Open the OWASP Juice Shop product search feature.
2. Capture the search request using Burp Suite.
3. Send the request to Burp Intruder and select the `q` parameter as the injection point.
4. Test the parameter with different XSS payloads.
5. Check the responses to identify a payload that is reflected by the application.
6. Open the successful payload in the browser.
7. Observe the JavaScript alert displayed by the browser. This confirms that the injected script was executed.

## Evidence

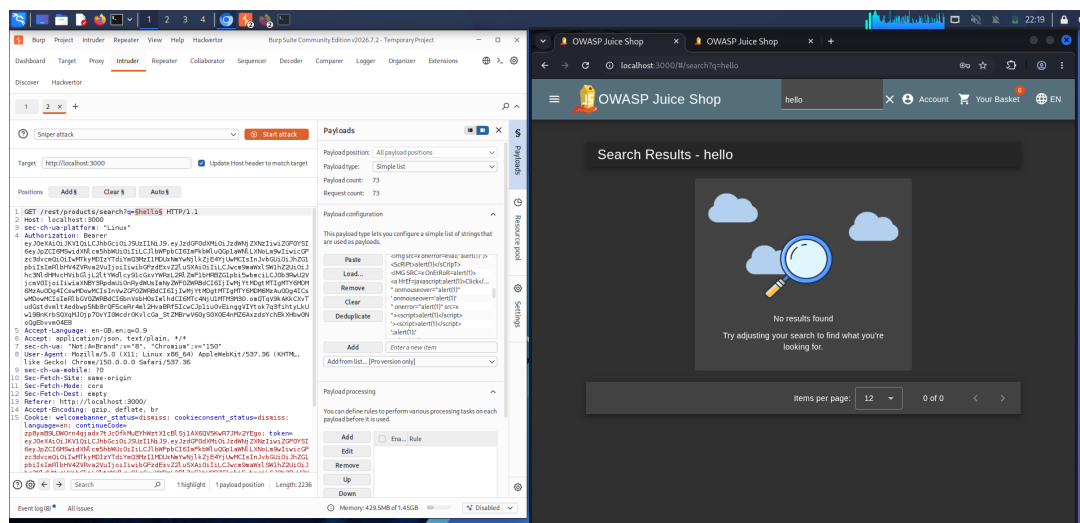


Figure 7.17: Burp Intruder testing XSS payloads against the q parameter

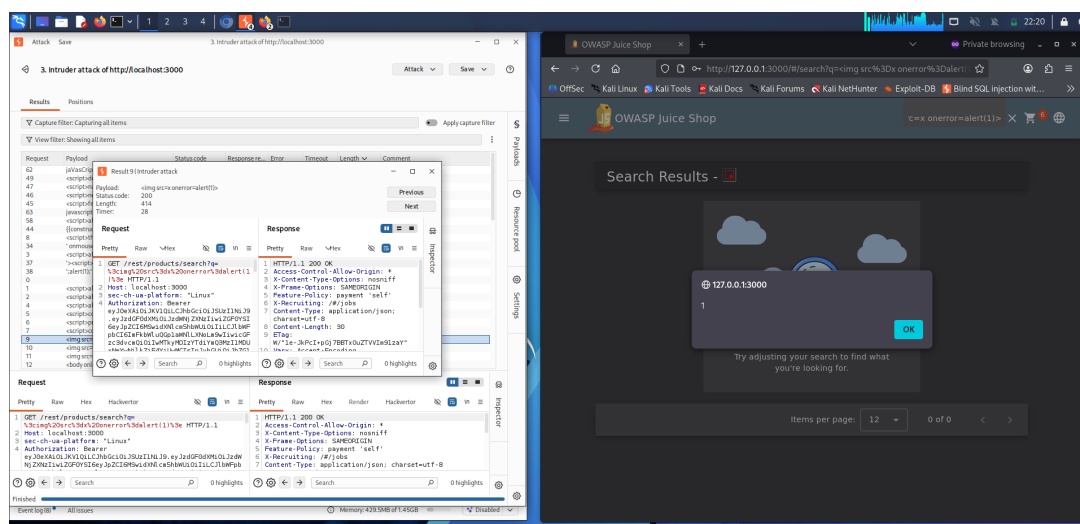


Figure 7.18: JavaScript alert confirming execution of the injected XSS payload

## Impact

An attacker can create a malicious search link containing JavaScript and send it to another user.

If the victim opens the link, the injected script can run in the context of the OWASP Juice Shop application.

Depending on the user's privileges and the application's security controls, this may allow the attacker to perform actions using the victim's browser, modify displayed content, redirect the victim to another page, or perform other unauthorized actions.

## Remediation

- Encode user-supplied input before displaying it on the webpage so that it is treated as text rather than executable JavaScript.
- Validate the `q` parameter on the server side and reject unexpected input where appropriate.
- Use secure output-encoding functions provided by the application's framework.
- Implement a strong Content Security Policy (CSP) as an additional layer of protection.
- Test all user-controlled input points for XSS before deploying the application.

## Conclusion

The Reflected XSS vulnerability in the product search parameter was successfully confirmed through manual testing using Burp Suite.

The injected payload was reflected by the application and executed in the browser when the crafted input was opened.

The finding is therefore classified as a confirmed Medium-severity Reflected Cross-Site Scripting vulnerability.

## 7.6 Cross-Site Request Forgery

### Cross-Site Request Forgery – Profile Update (CSRF)

|                           |                                             |
|---------------------------|---------------------------------------------|
| <b>Finding ID</b>         | CSRF                                        |
| <b>Severity</b>           | Medium                                      |
| <b>CVSS 3.1 Score</b>     | 4.3 (Medium)                                |
| <b>CVSS Vector</b>        | AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N         |
| <b>Status</b>             | Confirmed                                   |
| <b>OWASP Category</b>     | A01:2021 – Broken Access Control            |
| <b>CWE</b>                | CWE-352 – Cross-Site Request Forgery (CSRF) |
| <b>Affected Endpoint</b>  | POST <code>/profile</code>                  |
| <b>Affected Parameter</b> | <code>username</code>                       |

**Table 7.7:** CSRF finding details

## Description

The profile update functionality does not provide sufficient protection against Cross-Site Request Forgery (CSRF) attacks.

The profile update request does not contain a CSRF protection token. A malicious external HTML page can therefore create and automatically submit a request to the `/profile` endpoint while the victim is authenticated.

During testing, the username of the authenticated user was changed to an attacker-controlled value by opening the malicious HTML page.

## Methodology

The vulnerability was identified by intercepting the profile update request using Burp Suite.

The request was reviewed to determine whether a CSRF token or other effective CSRF protection mechanism was present.

A proof-of-concept HTML page was then created containing a form that submitted a POST request to the profile endpoint. The form was automatically submitted using JavaScript.

The test was performed while a user was authenticated to the application. The username was successfully changed to the value specified in the malicious form, confirming the CSRF vulnerability.

## Steps to Reproduce

1. Log in to the OWASP Juice Shop application using a valid user account.
2. Open the profile page and intercept the username update request using Burp Suite.
3. Observe that the profile update request does not contain a CSRF protection token.
4. Create a malicious HTML file containing the following form:

```
<form action="http://127.0.0.1:3000/profile" method="POST">
 <input name="username" value="CSRF"/>
</form>
<script>document.forms[0].submit();</script>
```

5. Save the file as `csrfjuice.html`.
6. While still authenticated to the Juice Shop application, open the malicious HTML file.
7. Observe that the form is automatically submitted to the `/profile` endpoint.
8. Return to the Juice Shop profile and observe that the username has been changed to the attacker-controlled value.

## Evidence

The following evidence demonstrates the missing CSRF protection, the proof-of-concept payload, and the successful execution of the attack.

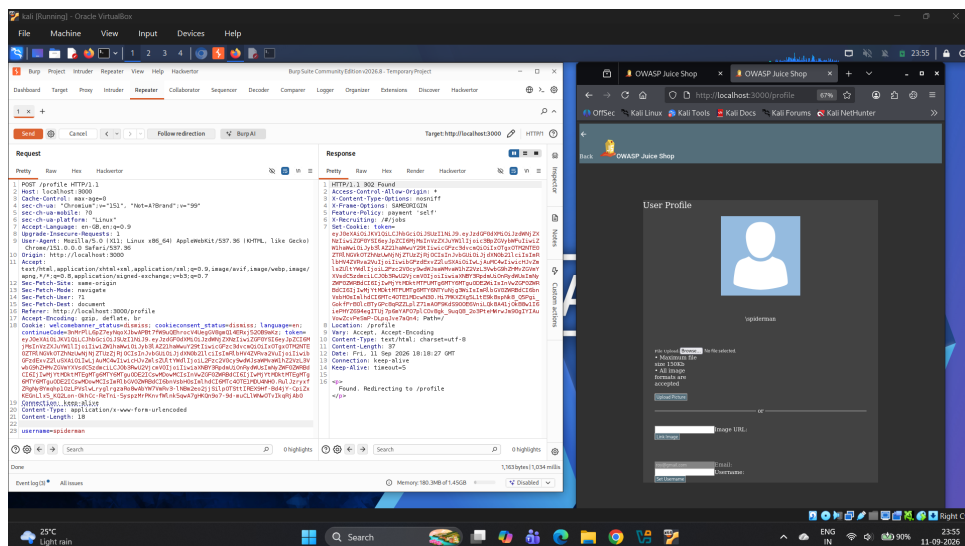


Figure 7.19: Burp Suite showing the profile update request without a CSRF token

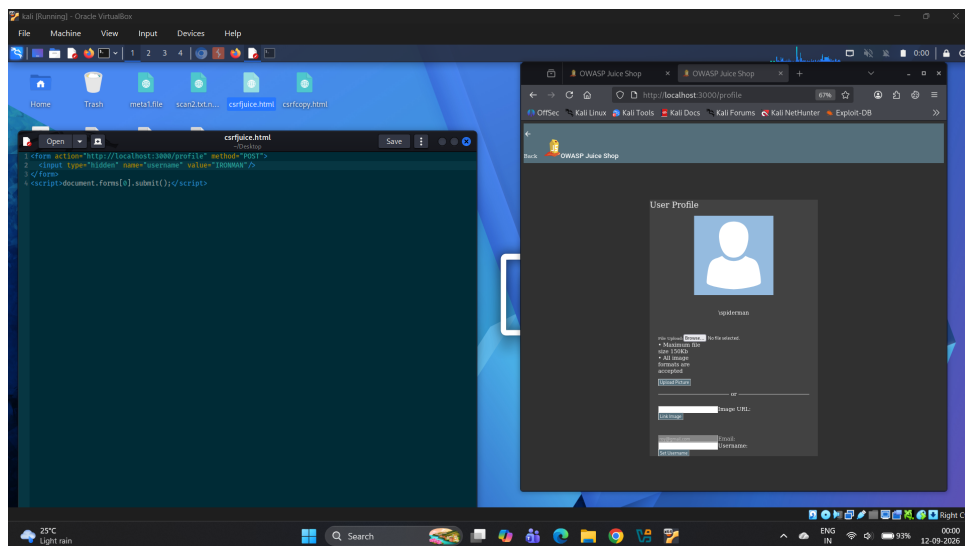
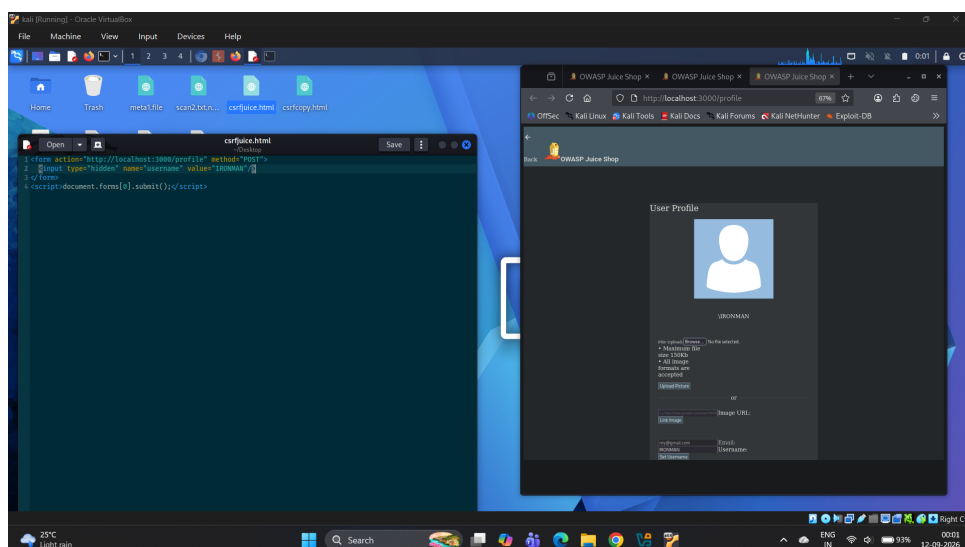


Figure 7.20: CSRF proof-of-concept HTML form used to submit the request automatically



**Figure 7.21:** Successful username change after the CSRF payload was executed

## Impact

An attacker can create a malicious webpage that submits an unwanted profile update request when an authenticated victim visits the page.

In this case, the attacker can change the victim's username without requiring the victim to manually submit the profile update form.

If similar CSRF protection is missing from other sensitive operations, the impact could be greater depending on the actions available to an authenticated user.

## Remediation

- Implement a strong, unpredictable CSRF token for state-changing requests.
- Validate the CSRF token on the server side before processing the request.
- Use appropriate SameSite cookie settings as an additional layer of protection.
- Apply CSRF protection consistently to all sensitive state-changing operations.
- Do not rely only on client-side validation for CSRF protection.

## Conclusion

The CSRF vulnerability was successfully confirmed through manual testing. A malicious HTML page was able to submit a profile update request while the victim was authenticated, resulting in an attacker-controlled username change.

The finding is therefore classified as a confirmed Medium-severity security issue.

## 7.7 Open Redirect

### Open Redirect – Unvalidated Redirect Parameter (OR)

Finding ID	OR
Severity	Medium
CVSS 3.1 Score	4.3 (Medium)
CVSS Vector	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N
Status	Confirmed
OWASP Category	A01:2021 – Broken Access Control
CWE	CWE-601 – URL Redirection to Untrusted Site (Open Redirect)
Affected Endpoint	GET /redirect
Affected Parameter	to

Table 7.8: Open Redirect finding details

#### Description

The application provides a redirect feature at `/redirect`, which takes a destination website through the `to` parameter and automatically sends the user's browser there.

The application attempts to allow redirects only to trusted destinations, but the validation check is not strict enough. Instead of properly verifying the actual destination, it checks whether a trusted domain string appears somewhere in the URL.

This allows the validation check to be bypassed by placing an attacker-controlled destination first, followed by a trusted domain string elsewhere in the same URL.

#### Methodology

The vulnerability was first identified during the OWASP ZAP automated scan. Manual testing was then performed to verify whether the redirect validation could be bypassed.

A trusted redirect destination was tested first, followed by an untrusted destination. Finally, a crafted URL containing an attacker-controlled domain and a trusted domain string was tested.

The browser was successfully redirected to the attacker-controlled domain, confirming the vulnerability.

#### Steps to Reproduce

1. Confirm that a known trusted destination redirects correctly:

```
http://127.0.0.1:3000/redirect?to=https://github.com/juice-shop/juice-shop
```

2. Attempt an untrusted destination directly and observe that it is blocked with a 406 `Unrecognized target URL` error:

```
http://127.0.0.1:3000/redirect?to=https://www.google.com
```

3. Bypass the validation check by placing an attacker-controlled domain first, followed by a trusted domain string elsewhere in the URL:

```
http://127.0.0.1:3000/redirect?to=https://evil.com/?https://github.com/juice-shop/juice-shop
```

4. Observe that the browser is redirected to `evil.com` instead of being blocked. This confirms that the redirect validation can be bypassed.

## Evidence

The following evidence demonstrates both automated detection and manual confirmation of the vulnerability.

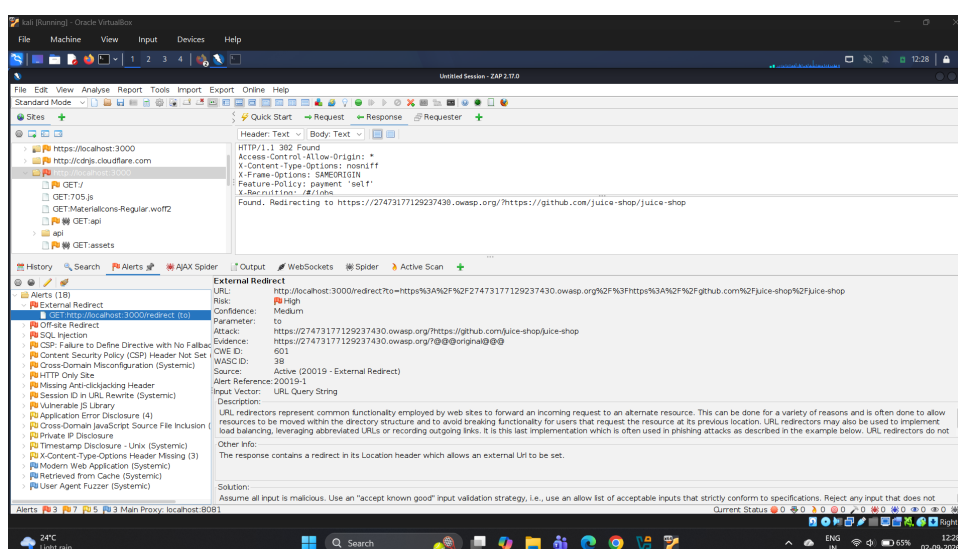


Figure 7.22: OWASP ZAP alert flagging the redirect parameter as vulnerable

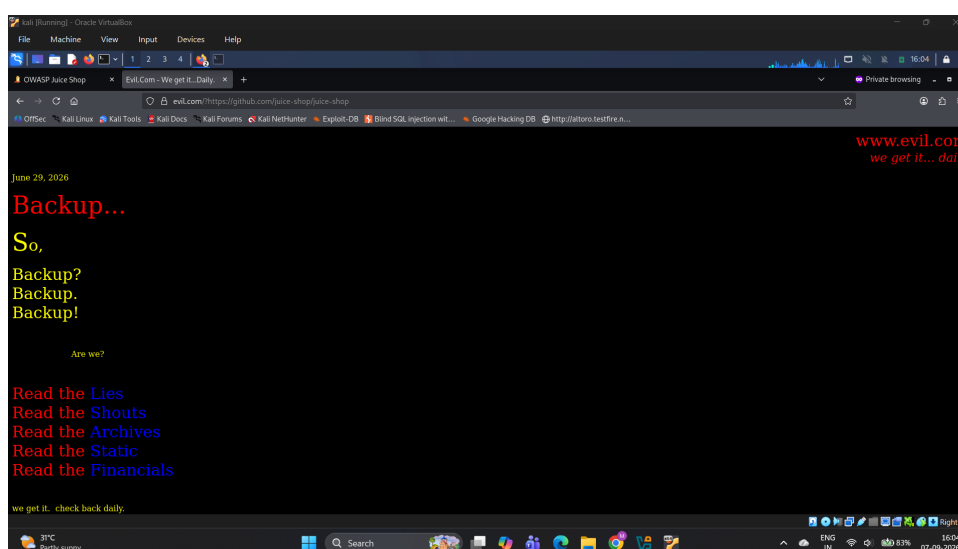


Figure 7.23: Manual confirmation of the bypass, redirecting to an untrusted domain

## Impact

An attacker can craft a link that begins with the trusted application domain but ultimately redirects the victim to an attacker-controlled website.

This can be used in phishing and social engineering attacks, where the attacker attempts to make the destination appear trustworthy.

Because the initial link uses the trusted application domain, a victim may be less likely to notice the malicious destination before following the link.

## Remediation

- Do not allow the client to control the full redirect destination through a URL parameter.
- If redirects to external sites are required, validate the destination against a strict allowlist of exact, complete domain names rather than using substring matching.
- Prefer using relative paths for internal redirects instead of accepting complete external URLs.
- Reject redirect destinations that do not match the approved allowlist.
- Consider displaying a warning page before redirecting to an external domain, allowing the user to confirm the destination.

## Conclusion

The Open Redirect vulnerability was successfully confirmed through manual testing. The application allowed an attacker-controlled destination to bypass the redirect validation and redirect the browser to an external domain.

The finding is therefore classified as a confirmed Medium-severity security issue.

## 8 Phase 5 – Post-Exploitation and Impact Analysis

This phase analyzes the potential consequences of the confirmed vulnerabilities when considered individually and in combination. The analysis focuses on demonstrated security impact as well as realistic attack-chain scenarios that could occur if the vulnerabilities were present in a production deployment handling real user data.

### Objective

Phase 4 confirmed the identified vulnerabilities through individual manual verification and proof-of-concept testing. Phase 5 considers how these confirmed weaknesses could interact with one another and how their combined impact could affect the application's confidentiality, integrity, and access-control mechanisms.

The attack-chain scenarios presented below represent potential combined impacts based on the confirmed vulnerabilities. They are not presented as additional exploitation activities unless specifically demonstrated during the assessment.

### Attack Chain Scenarios

#### Scenario 1: Unauthenticated Database Access via SQL Injection

The SQL Injection vulnerability in the product search endpoint (SQLI-02) does not require authentication. During testing, SQL Injection was used to identify the SQLite database structure and retrieve records from the `Users` table, including email addresses and password hashes.

In a production environment, unauthorized access to user records could expose sensitive account information. If users reused weak passwords or if the exposed password hashes were successfully cracked, the information could potentially be used in credential-stuffing attacks against other services.

The assessment demonstrated unauthorized database access through SQLI-02; credential cracking and attacks against external services were not performed as part of this assessment.

#### Scenario 2: Privilege Escalation to Administrative Access

The Broken Access Control finding (BAC) allows an unauthenticated visitor to register an account with the `admin` role. During testing, the newly created account successfully accessed the Administration page.

SQLI-01 provides a separate authentication-bypass path that was also demonstrated during testing. Therefore, the application contains multiple independent weaknesses that can result in unauthorized administrative access.

Once administrative access is obtained, the application's administration functionality may provide access to user data, product information, and other administration-only functionality. The demonstrated administrative access represents a significant failure of the application's authorization controls.

### **Scenario 3: Client-Side Impact via Reflected XSS and CSRF**

The Reflected XSS finding demonstrates that attacker-controlled JavaScript can execute in a victim's browser through a crafted search request.

The CSRF finding separately demonstrates that a cross-site request can cause a profile update while the victim is authenticated.

In combination, these weaknesses could increase the impact of a client-side attack by allowing attacker-controlled script execution and unauthorized state-changing requests under suitable conditions.

The assessment confirmed the XSS and CSRF vulnerabilities individually. A complete combined XSS-to-CSRF attack chain, including session-data extraction or other browser-side credential theft, was not separately demonstrated and is therefore considered a potential impact scenario rather than a confirmed exploitation result.

### **Scenario 4: Account Takeover via Weak Password Recovery**

The weak password recovery mechanism (AUTH) allows an attacker who can obtain, research, or correctly guess the relevant security- question answer to reset a target account's password.

During testing, the password recovery weakness was successfully demonstrated against the tested account. If the same weakness were present for an account with administrative privileges, successful exploitation could potentially provide unauthorized administrative access.

This represents an additional path to account compromise that is independent of the BAC and SQLI-01 findings.

### **Scenario 5: Unauthorized Data Exposure via IDOR**

The IDOR finding allows an authenticated user to view another user's basket contents by manipulating the basket identifier.

The assessment demonstrated unauthorized access to another user's basket data. In a production environment, automated enumeration of predictable or accessible identifiers could potentially increase the number of affected accounts and expose additional shopping-related information.

The assessment did not perform large-scale enumeration of basket identifiers; therefore, such broader data harvesting is considered a potential impact scenario.

## Consolidated Business Impact

If the confirmed vulnerabilities existed in a production application handling real customer data, the combined impact could include:

- **Unauthorized administrative access** through multiple independent weaknesses, including the demonstrated BAC and SQLI-01 attack paths and the potential impact of the AUTH weakness against privileged accounts.
- **Unauthorized access to user database records** through SQLI-02, including email addresses and password hashes retrieved during testing.
- **Client-side attacks against legitimate users** through Reflected XSS, with additional unauthorized state changes potentially possible when combined with the confirmed CSRF weakness.
- **Exposure of individual users' shopping and basket data** through IDOR.
- **Erosion of user trust** through Open Redirect, which could potentially be used to support phishing campaigns by redirecting users from the legitimate application domain to an attacker-controlled destination.

## Conclusion

The individual findings documented in Phase 4 are significant on their own, and several provide overlapping paths to high-value outcomes such as unauthorized administrative access or sensitive data exposure.

The assessment demonstrated multiple independent weaknesses affecting authentication and authorization controls. This indicates that remediation should address the underlying security controls rather than relying on a single isolated fix. For example, addressing SQLI-01 alone would not remove the administrative-access risk created by the BAC finding, while the AUTH weakness represents an additional potential path to account compromise.

Detailed remediation guidance for each individual finding, along with consolidated recommendations, is provided in Chapter 10.

## 9 Phase 6 – Reporting and Risk Assessment

### 9.1 Vulnerability Findings Overview

#### Confirmed Vulnerabilities

ID	Vulnerability	Severity	Status
BAC	Broken Access Control – Privilege Escalation via Registration	Critical	Confirmed
IDOR	Insecure Direct Object Reference – Unauthorized Basket Access	Medium	Confirmed
AUTH	Weak Password Recovery	Critical	Confirmed
SQLI-01	SQL Injection – Authentication Bypass	Critical	Confirmed
SQLI-02	SQL Injection – Product Search and Data Extraction	High	Confirmed
XSS	Reflected Cross-Site Scripting – Search Parameter	Medium	Confirmed
CSRF	Cross-Site Request Forgery – Profile Update	Medium	Confirmed
OR	Open Redirect – Unvalidated Redirect Parameter	Medium	Confirmed

**Table 9.1:** Summary of confirmed security vulnerabilities

#### Risk Overview

The assessment identified eight confirmed security vulnerabilities affecting access control, authentication, application input, client-side security, request integrity, and redirect handling.

Three findings were classified as Critical, one as High, and four as Medium.

The Critical findings include Broken Access Control, SQL Injection – Authentication Bypass, and Weak Password Recovery. The Broken Access Control finding affects privilege-related actions during registration. The Weak Password Recovery finding allows an attacker to take over an account, including a privileged account, through a predictable security question.

The single High-severity finding is SQL Injection – Product Search and Data Extraction,

which allows unauthorized database interaction without requiring authentication.

The Medium-severity findings include Reflected Cross-Site Scripting, Insecure Direct Object Reference, Open Redirect, and Cross-Site Request Forgery. The Reflected XSS vulnerability can allow attacker-controlled JavaScript to execute in a victim's browser when a malicious link is opened. The IDOR vulnerability allows access to another user's basket by changing the basket identifier in the request. The Open Redirect vulnerability allows an attacker-controlled destination to redirect a victim to an external website. The CSRF vulnerability allows an attacker-controlled webpage to submit a profile update request while a victim is authenticated.

## Finding Distribution

The confirmed findings cover the following major security areas:

- **Broken Access Control** – Unauthorized privilege escalation through the affected registration functionality.
- **Insecure Direct Object Reference** – Unauthorized access to another user's basket by manipulating the basket identifier.
- **Authentication** – Weakness in the password recovery mechanism through the security-question process.
- **SQL Injection** – Authentication bypass and unauthorized retrieval of application data.
- **Cross-Site Scripting** – Execution of attacker-controlled JavaScript in a victim's browser.
- **Cross-Site Request Forgery** – Unauthorized profile modification through a cross-site request.
- **Open Redirect** – Redirect validation weakness allowing navigation to an external domain.

Detailed testing steps, evidence, impact, and remediation recommendations for each confirmed finding are provided in the following sections of this report.

## 9.2 Risk Summary

This section provides an overall summary of the security risks associated with the vulnerabilities confirmed during the assessment of the OWASP Juice Shop application.

### Overall Risk Assessment

The assessment identified eight confirmed vulnerabilities across authentication, authorization, input validation, client-side security, request integrity, and redirect handling.

Three findings were classified as Critical, one as High, and four as Medium. The Critical and High severity findings affect important application security controls and should be prioritized for remediation.

The risk assessment presented in this report is based on the vulnerabilities that were identified and manually validated during the engagement.

## Severity Rating Definitions

Severity	Definition
Critical	Vulnerabilities with the potential to cause severe compromise of authentication, authorization, application functionality, or sensitive data, depending on the affected component and exploitation conditions.
High	Vulnerabilities that can result in significant unauthorized access, data exposure, account compromise, or execution of attacker-controlled content under the applicable exploitation conditions.
Medium	Vulnerabilities that weaken application security controls or enable unauthorized actions or further attacks, but generally have more limited impact or require additional conditions.
Low	Vulnerabilities with limited direct security impact, such as minor information disclosure or configuration weaknesses.

**Table 9.2:** Severity rating definitions used in this report

## Risk Distribution

Severity	Number of Findings	Percentage
Critical	3	37.5%
High	1	12.5%
Medium	4	50%
Low	0	0%
<b>Total</b>	<b>8</b>	<b>100%</b>

**Table 9.3:** Distribution of confirmed findings by severity

## Risk Matrix

Finding ID	Likelihood of Exploitation	Business Impact
SQLI-01	High	Critical
BAC	High	Critical
AUTH	Medium	Critical
SQLI-02	High	High
XSS	Medium	Medium
IDOR	High	Medium
OR	Medium	Medium
CSRF	Medium	Medium

**Table 9.4:** Risk matrix showing likelihood and business impact for each confirmed finding

## Key Risk Observations

- Three Critical-severity vulnerabilities were confirmed during the assessment. These findings affect authentication, authorization, and account recovery functionality.
- The BAC finding demonstrates a weakness in authorization controls that can allow unauthorized privilege-related actions through the affected registration functionality.
- SQLI-01 demonstrates that the login functionality can be bypassed through SQL Injection, resulting in unauthorized administrative access.
- The AUTH finding demonstrates that the password recovery mechanism relies on a predictable security question, allowing an attacker to take over the affected account without knowledge of the original password. If the same weakness affected a privileged account, it could potentially lead to unauthorized administrative access. This finding was classified as Critical severity based on its CVSS score and the potential for full account takeover.
- SQLI-02 was classified as High severity. It does not require authentication, but its confirmed impact was limited to unauthorized read access to database records rather than a full authentication bypass.
- The IDOR finding demonstrates that server-side authorization checks are insufficient for the affected basket resource, as the basket identifier can be manipulated to access another user's resource. This finding was classified as Medium severity based on the confidentiality-only impact and the requirement for an authenticated session.
- The XSS, CSRF, and Open Redirect findings demonstrate additional Medium-severity weaknesses affecting client-side execution, request integrity, and redirect handling respectively.
- No Low-severity vulnerabilities are included in the final confirmed findings because the assessment focused on findings that were successfully identified and manually validated with demonstrable security impact.

## Conclusion of Risk Assessment

The confirmed vulnerabilities demonstrate security weaknesses in multiple application controls, including authentication, authorization, input validation, client-side security, and request integrity.

The Critical and High severity findings should be addressed before the application is deployed in an environment where it processes real user data or provides access to sensitive functionality.

# 10 Phase 7 – Remediation and Retesting

## 10.1 Remediation Recommendations

This section consolidates the remediation recommendations for all confirmed vulnerabilities identified during this assessment. Detailed, finding-specific remediation guidance is also provided within each individual finding section earlier in this report.

### Prioritized Remediation Summary

Finding ID	Vulnerability	Priority	Recommended Fix
Sqli-01	SQL Injection – Login	Immediate	Use parameterized queries
BAC	Broken Access Control	Immediate	Enforce role assignment on server side
AUTH	Weak Password Recovery	Immediate	Use secure token-based password reset
Sqli-02	SQL Injection – Search	High	Use parameterized queries
XSS	Reflected XSS	Medium	Encode output and apply CSP
IDOR	Insecure Direct Object Reference	Medium	Verify object ownership server-side
OR	Open Redirect	Medium	Validate redirect destination with strict allowlist
CSRF	Cross-Site Request Forgery	Medium	Implement CSRF tokens

**Table 10.1:** Prioritized remediation summary

## Immediate Priority – Critical Findings

### SQL Injection – Authentication Bypass (SQLI-01)

- Use parameterized queries or prepared statements for all database queries, instead of directly inserting user input into SQL statements.
- Apply appropriate server-side validation to the `email` parameter.
- Use a database account with only the minimum privileges required by the application.
- Prevent database errors and SQL query details from being exposed to application users.

### Broken Access Control (BAC)

- Do not allow the client to control the `role` value during account registration.
- Always assign the default user role on the server side, regardless of any value supplied by the client.
- Restrict administrator privilege assignment to a separate, authenticated, and authorized administrative process.
- Apply server-side validation to reject unexpected or unauthorized fields in the registration request.

### Weak Password Recovery Mechanism (AUTH)

- Replace security-question-based recovery with a cryptographically random, single-use, time-limited reset token sent through a verified communication channel.
- Apply rate limiting and monitoring to password recovery attempts.
- Avoid revealing information that helps an attacker determine whether a target account exists.
- Store passwords using a strong, modern password hashing algorithm such as Argon2 or bcrypt with appropriate salting.

## High Priority Findings

### SQL Injection – Product Search Parameter (SQLI-02)

- Use parameterized queries or prepared statements for the product search query, instead of directly inserting the `q` parameter into SQL statements.
- Apply appropriate server-side validation to the `q` parameter.
- Use a database account with only the minimum privileges required by the application.
- Prevent database errors and SQL query details from being exposed to application users.

## Medium Priority Findings

### Reflected Cross-Site Scripting (XSS)

- Encode all user-supplied input before displaying it back on the page.
- Apply appropriate context-aware output encoding to the `q` parameter before rendering user-controlled content.
- Implement a Content Security Policy (CSP) to restrict which scripts are allowed to execute on the page.

### Insecure Direct Object Reference (IDOR)

- Verify on the server side that the requested resource (e.g. basket) belongs to the currently authenticated user before returning any data.
- Reject the request with an access-denied response if ownership cannot be verified.
- Avoid exposing predictable identifiers for resources containing user-specific data where appropriate.

### Open Redirect (OR)

- Validate the redirect destination against a strict allowlist of approved destinations rather than checking for a substring match.
- Prefer relative paths for internal redirects instead of accepting a full external URL from the client.
- Where external redirects are required, consider displaying a clear warning before navigating to an external domain.

### Cross-Site Request Forgery (CSRF)

- Implement a strong, unpredictable CSRF token for all state-changing requests.
- Validate the CSRF token on the server side before processing any request.
- Apply appropriate `SameSite` cookie settings as an additional layer of protection.

## General Security Recommendations

In addition to the finding-specific fixes above, the following general improvements are recommended to strengthen the overall security posture of the application:

- Apply the principle of least privilege consistently across all database accounts, user roles, and application components.
- Perform regular security testing, including both automated scanning and manual validation, as part of the development lifecycle.

- Apply consistent server-side validation across all endpoints, rather than relying on client-side checks alone.
- Implement centralized security logging and monitoring to detect and respond to suspicious activity.
- Review and configure missing or incomplete security headers, such as the Content-Security-Policy header, across all application routes.

## 10.2 Retesting

This section describes the retesting approach that will be used to verify the remediation of vulnerabilities identified during the assessment.

Retesting should be performed after the recommended security fixes have been implemented. The purpose of retesting is to determine whether the previously identified vulnerabilities have been successfully addressed and whether the implemented fixes introduce any new security issues.

### Retesting Objectives

The main objectives of the retesting activity are:

- Verify that the previously confirmed vulnerabilities have been remediated.
- Reproduce the original test conditions and verify that the previously successful proof-of-concept no longer works.
- Confirm that the implemented security controls are enforced correctly on the server side.
- Verify that remediation does not negatively affect the intended application functionality.
- Identify any vulnerabilities that remain partially or completely unresolved.

### Retesting Methodology

The retesting process will follow the same general approach used during the original assessment.

1. Review the remediation implemented for each confirmed vulnerability.
2. Reproduce the original request, input, or attack condition used during the initial assessment.
3. Verify whether the previously demonstrated exploitation remains possible.
4. Verify that the relevant security control is now correctly enforced.
5. Test the affected application functionality to confirm that the remediation has not introduced unintended behavior.
6. Record the retesting result and supporting evidence for each vulnerability.

## Retesting Status

At the time of preparing this report, remediation had not yet been implemented against the target application. Retesting was therefore not performed as part of this engagement and is documented here as planned future work, to be carried out once the recommended fixes in Section 10.1 have been applied.

The following table defines the retesting status that will be used for the confirmed findings once remediation is complete:

Finding ID	Vulnerability	Status
BAC	Broken Access Control – Privilege Escalation via Registration	Pending
IDOR	Insecure Direct Object Reference – Unauthorized Basket Access	Pending
AUTH	Weak Password Recovery	Pending
SQLI-01	SQL Injection – Authentication Bypass	Pending
SQLI-02	SQL Injection – Product Search and Data Extraction	Pending
XSS	Reflected Cross-Site Scripting – Search Parameter	Pending
CSRF	Cross-Site Request Forgery – Profile Update	Pending
OR	Open Redirect – Unvalidated Redirect Parameter	Pending

**Table 10.2:** Retesting status of confirmed vulnerabilities

## Retesting Evidence

For each remediated vulnerability, evidence should be collected to demonstrate the result of the retest.

The evidence may include:

- HTTP requests and responses captured using Burp Suite.
- Screenshots showing the security control preventing the previously demonstrated attack.
- Updated application behavior after remediation.
- Scanner results, where automated verification is applicable.
- Any remaining error messages or security-related responses.

## Retesting Result Classification

Each vulnerability should be assigned one of the following results after retesting:

- **Remediated** – The vulnerability could no longer be reproduced and the relevant security control was verified.
- **Partially Remediated** – The original exploitation was reduced or restricted, but the vulnerability or its security impact remains under certain conditions.
- **Not Remediated** – The original vulnerability could still be reproduced after remediation.
- **Not Retested** – Retesting could not be performed because the required remediation or testing conditions were not available.

### Final Retesting Statement

Retesting results will be updated after the recommended remediation activities have been implemented and verified. As this engagement was conducted against an intentionally vulnerable training application rather than a production system, remediation and retesting fall outside the scope of the current assessment and are recommended as a follow-up activity.

The final retesting record should include the original finding ID, remediation status, retesting procedure, observed result, supporting evidence, and final status for each vulnerability.

This approach provides a clear comparison between the original security assessment and the post-remediation security state of the application.

# 11 Conclusion

This Vulnerability Assessment and Penetration Testing (VAPT) engagement was conducted against the OWASP Juice Shop application in a controlled local testing environment. The assessment followed a structured methodology consisting of planning and scoping, reconnaissance, vulnerability scanning and assessment, and manual verification of identified vulnerabilities.

During the assessment, automated security testing and manual testing were used to identify and validate security weaknesses in the application. The identified vulnerabilities were manually verified and supported with appropriate evidence collected during the assessment.

## Summary of Findings

A total of eight security vulnerabilities were confirmed during the assessment. The findings covered multiple security areas, including access control, authentication, SQL injection, cross-site scripting, request integrity, and redirect handling.

The confirmed findings are summarized below:

- **BAC – Broken Access Control:** Privilege escalation through the registration functionality.
- **IDOR – Insecure Direct Object Reference:** Unauthorized access to another user's basket by manipulating the basket identifier.
- **AUTH – Weak Password Recovery:** Weaknesses in the password recovery mechanism that may affect account security.
- **Sqli-01 – SQL Injection:** SQL injection affecting the login functionality and allowing authentication bypass under the tested conditions.
- **Sqli-02 – SQL Injection:** SQL injection affecting the product search functionality and allowing unauthorized database interaction under the tested conditions.
- **XSS – Reflected Cross-Site Scripting:** Execution of attacker-controlled JavaScript through the vulnerable search functionality.
- **CSRF – Cross-Site Request Forgery:** Unauthorized profile modification through a cross-site request while the victim is authenticated.
- **OR – Open Redirect:** Redirect handling that allows an attacker-controlled destination to be used under the tested conditions.

## Risk Summary

The confirmed findings consisted of three Critical-severity vulnerabilities, one High-severity vulnerability, and four Medium-severity vulnerabilities.

The Critical findings affect important application security controls, including authentication, authorization, and account recovery. The High and Medium findings demonstrate additional weaknesses affecting database interaction, client-side security, resource authorization, request integrity, and redirect handling.

The combination of these findings demonstrates that multiple layers of application security controls require strengthening.

## Recommended Next Steps

The Critical and High severity findings should be addressed with priority. Recommended remediation measures include implementing parameterized database queries, enforcing server-side authorization, strengthening password recovery, applying appropriate output encoding, implementing object-level access controls, and protecting state-changing requests against CSRF.

Following remediation, the application should undergo a dedicated retesting process. Each confirmed vulnerability should be tested again using the original proof-of-concept and relevant alternative test cases to verify that the vulnerability has been effectively addressed.

## Limitations

This assessment was conducted under the following constraints, which should be considered when interpreting the findings in this report:

- The engagement was performed by a single tester within a fixed testing window, rather than a multi-tester team over an extended engagement period.
- Testing was scoped to the locally hosted OWASP Juice Shop instance only; no testing was performed against a production deployment, third-party services, or the underlying host operating system.
- Denial-of-Service (DoS) testing was explicitly excluded from the assessment scope and was not performed.
- Automated scanning coverage was limited to OWASP ZAP, ffuf, and SQLMap; findings not detected by these tools or through manual testing within the assessment window may not be reflected in this report.
- Large-scale exploitation activities, such as bulk data extraction, mass account enumeration, or credential-cracking against retrieved password hashes, were intentionally not performed, consistent with a controlled and non-disruptive testing approach.

This assessment reflects the security posture of the application at the time of testing and within the defined scope. It should not be interpreted as a guarantee that no additional vulnerabilities exist outside the tested scope or testing methodology.

## Final Statement

This assessment provides a security-focused view of the OWASP Juice Shop application based on the testing activities performed within the defined assessment scope. The findings, evidence, risk assessment, and remediation recommendations documented in this report provide a basis for improving the application's security controls.

Successful remediation should be followed by security retesting and continuous security testing throughout the application development lifecycle. This helps ensure that previously identified vulnerabilities remain resolved and that newly introduced security weaknesses are identified before deployment.